SIMATS ENGINEERING

Department of Computer Science and Engineering

DATA STRUCTURES

ASSIGNMENT 1 – CO1

Design, Analyse and Validate Data Structures for a Smart Library Management System

CO1: Understand the abstract properties of various linear and non-linear data structures such as Stacks, queues, lists, trees and graphs.

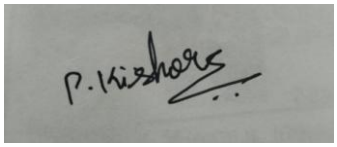
Student Name	**Kishore P**	**Register Number**	**192511046**
Department / Section	Computer Science and Engineering (CSE)	**Academic Year / Semester**	Sep–Dec 2026
Faculty Name	Dr. Supriya L P	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	06/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	**Task Title**	**Max Marks**	**Marks Awarded**	**Remarks**
1	Understand the Problem; Identify Data and Operations	4		Entities, relationships, operations with frequency and speed needs, and correctness rules identified. m-copies vs shared ADT compared.
2	Classify the Data Structures	3		Primitive vs non-primitive, linear vs non-linear, static vs dynamic justified. Flat array, 2-D array and per-level access compared.
3	Define and implement an ADT (WaitList)	5		Operations with pre/post conditions; correct insert/delete/print; boundary tests; array vs linked list; .h/.c separation.
4	Analyse Array Storage	4		Address formulas with worked examples; predicted output reasoned step by step; array kinds and parameter decay; O(n) middle insert/delete and doubling.
5	Analyse Recursive Solutions	4		Output predicted with trace; recurrences for show and mystery; 11 vs 19 calls; stack space and loop comparison.

Task	Task Title	Max Marks	Marks Awarded	Remarks
6	Count the Steps of an Algorithm	4		Step counts for four loops (136, 4, 64, 4) derived; notation explained; growth table; maintained free count.
7	Analyse Best, Average and Worst Cases	4		Conditions, counts and reasons for every case; average (n+1)/2 derived; binary search needs sorted data; O(n) cost to keep order.
8	Analyse Space Complexity	3		int array vs bitmap bytes calculated; recursion stack space compared; sizeof confirmed.
9	Implement and Test the Library Data Structures	6		Working init/issue/return/lookup with maintained count; tests for full, empty, first/last slot and 10-event sequence; expected vs actual output.
10	Compare Theory with Measurement	4		Same compiler flags and repetitions; real (not invented) timings; measured growth compared with Θ(n); counting vs clock() discussed.
11	Recommend and Defend the Final Design	5		Design for 150,000 copies with each component justified; operation complexities; load factor; trade-offs; evidence-based defence.
12	Re-evaluate and Adapt the Design	4		Impact of priority reservations, search and overdue-fine requirements; O(n) scan evaluated; revised function tested; index trade-offs; no unmeasured claims.
	TOTAL	50		Percentage :

OVERALL REMARKS



Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the “Evidence Expected” of each task.

Task	Evidence Expected	Status	Note
1	Problem definition, entities and relationships, operations, frequency requirements, correctness rules, and shared ADT versus independent copies.		
2	Classification of primitive/non-primitive, linear/non-linear, static/dynamic data structures; comparison of flat arrays, 2-D arrays and per-level arrays.		
3	ADT specification, preconditions and postconditions, C implementation of init, insert, delete and print, boundary tests, and .h/.c separation.		
4	1-D and row-major 2-D array address calculations, worked examples, output prediction, array parameter decay, insertion/deletion complexity and dynamic resizing.		
5	Recursive output traces, recurrence relations, complexity analysis of show and mystery, linear versus split count, and stack-space comparison.		
6	Step counts for four loops, derivation of operation counts, explanation of O, Ω and Θ, growth table, and maintained free-count analysis.		
7	Best-, average- and worst-case analysis, comparison counts, average-case derivation, binary-search requirements and the cost of keeping data sorted.		
8	Array-versus-bitmap memory calculations, recursion stack-space analysis, bit-operation complexity, sizeof verification, and working implementation with C cross-check.		
9	Working library implementation, init, issue, return and lookup, free-count invariant, boundary tests, ten-event sequence, expected-versus-actual outputs and complexity analysis.		
10	Repeated performance measurements, compiler flags, actual timing tables, comparison with theoretical growth, and discussion of operation counting versus clock().		
11	Recommended design for 150,000 copies, component justification, operation complexities, hash-table sizing at load factor 0.75, alternative comparisons and experimental evidence.		
12	Analysis of priority reservations, search and overdue-fine tracking; evaluation of the original design; revised implementation and tests; heap/index trade-offs; evidence-based defence.		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 8–9)

Evidence item	Status	Observation
Source code		

Evidence item	Status	Observation
Test inputs		
Expected behaviour		
Actual output		
Intermediate algorithmic evidence		
Boundary / failure tests		
Interpretation		
Code, data and report consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status	Note
All 12 tasks (Tasks 1–12) answered	☐Yes ☐Partly ☐No	
Problem definition and data-structure classifications (Tasks 1–2)	☐Yes ☐Partly ☐No	
ADT specification, implementation and boundary tests (Task 3)	☐Yes ☐Partly ☐No	
Array calculations and worked examples (Task 4)	☐Yes ☐Partly ☐No	
Recurrence derivations and call-stack traces (Task 5)	☐Yes ☐Partly ☐No	
Step counts and growth table (Task 6)	☐Yes ☐Partly ☐No	
Best, average and worst-case analysis (Task 7)	☐Yes ☐Partly ☐No	
Space-complexity calculations and sizeof check (Task 8)	☐Yes ☐Partly ☐No	
Working library implementation and test outputs (Task 9)	☐Yes ☐Partly ☐No	
Actual measurement tables and theory comparison (Task 10)	☐Yes ☐Partly ☐No	
Final design justified by analysis and evidence (Task 11)	☐Yes ☐Partly ☐No	
Adaptation to new requirements (Task 12)	☐Yes ☐Partly ☐No	
Every referenced figure, table and output is included	☐Yes ☐Partly ☐No	
Code, input data, output and written analysis agree	☐Yes ☐Partly ☐No	

DATA STRUCTURES

ASSIGNMENT 1 – CO1

CO1: Understand the abstract properties of various linear and non-linear data structures such as Stacks, queues, lists, trees and graphs

Nature of Assignment : Individual
Duration : 1 Week
Total Marks : 50
Submission Date : 06/11/2026

Assignment Title: Smart Library Management System (50 Marks)

Duration: 1 week

Design Challenge:

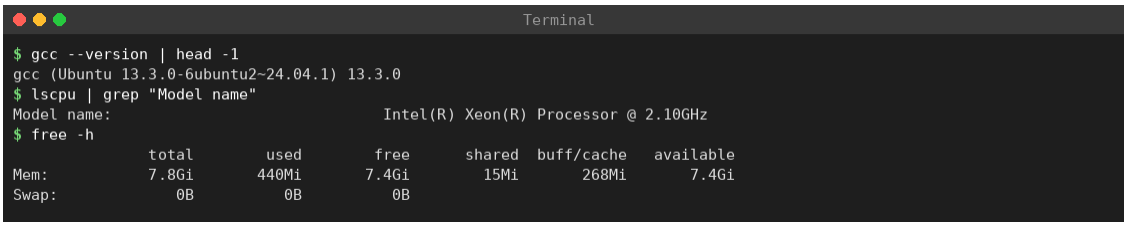
A university library holds 150,000 book copies spread across multiple floors, sections, shelves and racks. The system must support issuing and returning books, looking up a book by its ID, tracking available and borrowed copies, maintaining a waiting list when a requested book is already issued, and responding efficiently as the collection grows over the years.

Design and evaluate appropriate data structures for the Smart Library Management System. Analyse storage representation, operation costs, recursion, time and space complexity, and performance using the methods and experiments specified in Tasks 1–10. Compare arrays, dynamic arrays and linked lists for storing book records, and justify the choice of structures for the member borrowing lists, the reservation queue and the transaction history. Evaluate linear and binary search for book lookup, including the cost of keeping the records sorted, and implement recursive and iterative versions of the key operations to compare their call stack usage and space requirements. Back your theoretical analysis with performance experiments in C on datasets of increasing size and different orderings, and explain any difference between the predicted and measured results.

Finally, recommend and defend a suitable design for 150,000 copies, then re-evaluate it when new requirements are introduced: priority reservations for faculty and research scholars, search by title, author or subject as well as by ID, and overdue fine tracking in which the most overdue records must be retrieved first. Determine whether the original design should be retained, modified or redesigned, and justify the decision using analysis and evidence.

Note: the question paper's title line still reads "Smart Parking Management System"; this report follows its design challenge (Smart Library Management System).

Environment used for all programs: gcc 13.3.0 on Linux (Intel Xeon @ 2.10 GHz, 2 CPUs, 7.8 GiB RAM). Strict flags `-std=c11 -Wall -Wextra -pedantic` were used; every program compiled with **no warnings**.



Screenshot 1: Compiler and machine details (gcc version, CPU model, memory)

TASK 1 – Understand the Problem and Identify the Data and Operations (4 Marks)

To study the Smart Library Management System (SLMS), list every item of data it must store, every operation it must perform, how often each operation occurs, and which correctness rules must always hold. This analysis is done before any data structure is chosen, because a data structure is only "good" relative to the operations performed on it.

Problem summary

The library has several floors; each floor has sections, shelves and racks. The largest scenario in this assignment is **150,000 book copies**. Members (students, research scholars, faculty) borrow copies, return them, and wait in a queue when every copy of a title is issued. Overdue loans attract a fine. Display boards and the OPAC (catalogue terminals) read availability continuously.

Data to be stored

Entity	Attributes	Key	Remarks
Book (title record)	book ID, title, author, subject, number of copies	book ID	Many titles; searched by ID, title, author, subject
Copy	copy ID, book ID, status (available / issued), location (floor, shelf, position)	copy ID	About 150,000 records; read and updated most often
Member	member ID, category (student / scholar / faculty), borrowing list	member ID	Borrowing list is short (a few books) and changes often
Loan / transaction	loan ID, copy ID, member ID, issue date, due date, return date	loan ID	Created on issue, closed on return; history is append-only
Reservation	member ID, book ID, class (faculty / scholar / student), arrival number	book ID + arrival	Waiting list per title
Fine	loan ID, days overdue, rate, amount	loan ID	Computed from due date and return/today date

Relationships between the data

- Each issued copy belongs to exactly one member, and one loan record connects exactly one copy to one member (one-to-one while the loan is open).
- A member may hold several copies at once (one-to-many); a book has many copies (one-to-many).
- A reservation refers to one member and one book title; a fine refers to one loan.

Operations required

Operation	What it does	Assumed frequency	Speed need
Issue book	Picks an available copy, creates a loan	High	Fast

Operation	What it does	Assumed frequency	Speed need
Return book	Frees the copy, closes the loan, computes fine	High	Fast
Lookup by ID	Finds a copy/book record from its ID	Very high	Fast, O(1) average
Availability count	Shows available copies of a title / floor	Very high (display boards)	O(1)
Add to waiting list	Queues a member for an issued title	Medium	O(1)
Serve waiting list	Gives a returned copy to the next waiting member	Medium	O(1) / O(log n)
Overdue retrieval	Finds the most overdue loan	Daily / on demand	Fast
Search by title/author/subject	Catalogue search	High	Fast
Monthly report	Totals of fines and circulation	Once a month	May be slow

Correctness rules (invariants)

1. **No double issue:** one copy can never be issued to two members at once.
2. **Count correctness:** the displayed available count always equals the number of copies whose status is “available”.
3. **Waiting-list order:** members are served in arrival order within the same class; faculty/scholar priority applies only after Task 12.
4. **Validation first:** member validity and borrowing limit are checked before a copy is issued.
5. **Correct fine:** a fine uses the true due date and true return date of the same loan.

Alternative considered: m separate copies versus one shared ADT

Suppose each of m subsystems (circulation desk, OPAC, display board, fine module) keeps its own copy of the n copy records. Space becomes O(m·n) and every issue or return must update all m copies, so one missed update makes the copies disagree. One shared Abstract Data Type with a single copy needs O(n) space and has one place where updates are made.

Design space (n = 150,000, m = 4, 4-byte int status)	Bytes	Updates per issue	Risk
m independent copies	4 × 600,000 = 2,400,000 B (≈ 2.29 MiB)	4	Copies may disagree
One shared ADT	600,000 B (≈ 585.9 KiB)	1	Single point to protect

Conclusion

Copy records are addressed mainly by ID, the most frequent operations are update by ID, count and lookup, and the strict invariants favour **one shared structure with controlled access functions**. These findings drive Tasks 2–12.

TASK 2 – Classify the Data Structures (3 Marks)

To classify the data items and candidate structures of the SLMS as primitive or non-primitive, linear or non-linear, and static or dynamic, and to compare ways of reaching a copy by (floor, shelf, position).

Primitive data types

Data item	C type	Reason
Copy ID, member ID, loan ID, counts	int / size_t	Whole numbers; size_t for sizes and indexes
Member / copy ID text (e.g. “LB07C000123”)	char array	Fixed maximum length (15 + terminating \0)
Fine amount	double (or integer paise)	Fractional currency values
Status, member category	enum	Small fixed set: AVAILABLE, ISSUED; STUDENT, SCHOLAR, FACULTY

Non-primitive data structures

Structure	Class	Use in SLMS
struct	Non-primitive, user defined	Groups attributes of Book, Copy, Member, Loan
Array	Linear, static	Copies of one shelf stored by position
Dynamic array	Linear, grows at run time	Transaction history (append-only); floor with unknown capacity
Linked list	Linear, dynamic	Member borrowing list (short, frequent insert/delete)
Queue	Linear, FIFO	Reservation (waiting) list of a title
Stack	Linear, LIFO	Free-copy-slot stack; undo of last transaction
Hash table	Non-linear / indexed	Copy ID / title / author / subject → record
Tree / heap	Non-linear, hierarchical	Library → floors → shelves → copies; max-heap for most overdue loan; priority queue for reservations

Linear structures store elements one after another (array, list, queue, stack). Non-linear structures store elements in a hierarchy or by computed position (tree, heap, hash table).

Static versus dynamic

- **Static array:** capacity fixed at compile time; fastest, but wastes space if too large and fails if too small.
- **Dynamic array:** capacity chosen or changed at run time with `malloc`/`realloc` (doubling); needs `free`.

- **Linked list:** one node per element, allocated when needed; no resizing, but an extra pointer per node and no direct indexing.

Reaching a copy by (floor, shelf, position)

Assumed layout: 5 floors × 300 shelves per floor × 100 copies per shelf = 150,000 copies. With a flat array of equal-sized floors:

```
index = floor × 30,000 + shelf × 100 + position
```

Example: floor 2, shelf 15, position 7 → 2 × 30,000 + 15 × 100 + 7 = **61,507**. This is a few multiplications and additions, so access is O(1). A 2-D/3-D array gives the same O(1) access.

Representation	Access time	Unequal floors?	Comment
Flat array	O(1)	No (wastes space)	Simple and cache-friendly
3-D array a[floor][shelf][pos]	O(1)	No (rectangular)	Readable; same limitation
Array of per-floor dynamic arrays	O(1)	Yes	Each floor sized separately
Tree (library → floor → shelf)	O(depth)	Yes	Natural hierarchy; slower access

Member borrowing list, reservation queue, transaction history – choice

Collection	Operations	Best structure	Reason
Member borrowing list	add/remove a loan, short list	Singly linked list (or small array)	Few items; deletion without shifting
Reservation queue	add at rear, serve at front	Queue (linked list or circular array)	FIFO, O(1) both ends
Transaction history	append only, read in order	Dynamic array	O(1) amortised append, cache-friendly

Conclusion

Primitive types hold single values; structures combine them into records; arrays give constant-time access by index; hash tables index copies by ID; heaps order loans by overdue days. For equal-sized floors a flat array is most efficient; for unequal floors use per-floor arrays.

TASK 3 – Define and Implement an Abstract Data Type (5 Marks)

To define an Abstract Data Type **WaitList** that stores the ordered IDs of members waiting for a book, state its operations with preconditions and postconditions, implement it in C using an array, test it, and compare it with a linked-list implementation.

Theory: ADT and data structure

An **Abstract Data Type** is a mathematical description of a set of values and the operations allowed on them, written without saying how the values are stored ("what can be done", not "how"). A **data structure** is the concrete way the values are stored in memory together with the algorithms that implement the operations.

Aspect	ADT	Data structure
Viewpoint	Logical (what)	Physical (how)
Describes	Values, operations, rules	Memory layout and algorithms
Example	WaitList: ordered list of member IDs with insert/delete	Array with size counter, or singly linked list
Can change without affecting callers?	No – it is the contract	Yes – if the contract is kept

Why use an ADT in the library? Information hiding (callers use `list_insert`/`list_delete`, never `member[i]` directly); replaceability (array → linked list changes only the implementation file); testability (each operation has a precondition and postcondition); a single shared copy (Task 1).

Specification of WaitList

Values: a finite sequence of integers (member IDs) $m_0, m_1, \ldots, m_{(n-1)}$ with $0 \le n \le$ CAPACITY. The order is meaningful (arrival / priority order). CAPACITY = 8.

Operation	Signature	Precondition	Postcondition	Return
init	void list_init(WaitList *l)	l is not NULL	size = 0	–
insert	int list_insert(WaitList *l, int pos, int value)	0 ≤ pos ≤ size and size < CAPACITY	value at index pos; later elements moved right; size + 1	1 on success, 0 if a precondition fails (list unchanged)
delete	int list_delete(WaitList *l, int pos)	0 ≤ pos < size	element at pos removed; later elements moved left; size − 1	1 on success, 0 otherwise
print	void list_print(const WaitList *l)	none	list unchanged; elements and size printed	–

Operation	Signature	Precondition	Postcondition	Return
size	int list_size(const WaitList *l)	none	list unchanged	number of elements

Representation invariants: (1) 0 ≤ size ≤ CAPACITY; (2) only `member[0]` … `member[size-1]` are meaningful; (3) the relative order of elements not involved in an operation never changes.

Design decisions: array representation (small bounded list, simple to verify); status returned as int (1/0) so the caller decides how to react to an error; `const` pointer for print/size; the list is passed by pointer so the caller's object is modified and the array is not copied.

Algorithms (pseudocode)

```
Insert(pos, value):                    Delete(pos):
  if size = CAPACITY or pos < 0          if pos < 0 or pos >= size: return 0
     or pos > size: return 0             for i = pos to size-2:
  for i = size down to pos+1:               a[i] = a[i+1]
      a[i] = a[i-1]                      size = size - 1; return 1
  a[pos] = value; size = size + 1
  return 1
```

The loop in insert runs backwards so no value is overwritten before it is copied; the loop in delete runs forwards for the same reason.

Complete program

Program: t3.c

```c
/* t3.c - WaitList ADT (ordered member IDs waiting for a book), array version */
#include <stdio.h>

#define CAPACITY 8

typedef struct { int member[CAPACITY]; int size; } WaitList;

/* post: size == 0 */
void list_init(WaitList *l) { l->size = 0; }

/* pre: 0 <= pos <= size and size < CAPACITY; returns 1 on success, 0 otherwise */
int list_insert(WaitList *l, int pos, int value) {
    if (l->size == CAPACITY || pos < 0 || pos > l->size) return 0;
    for (int i = l->size; i > pos; --i) l->member[i] = l->member[i - 1]; /* shift right */
    l->member[pos] = value;
    ++l->size;
    return 1;
}

/* pre: 0 <= pos < size; returns 1 on success, 0 otherwise */
```

```c
int list_delete(WaitList *l, int pos) {
    if (pos < 0 || pos >= l->size) return 0;
    for (int i = pos; i < l->size - 1; ++i) l->member[i] = l->member[i + 1]; /* shift left *
/
    --l->size;
    return 1;
}

void list_print(const WaitList *l) {
    for (int i = 0; i < l->size; i++) printf("%d ", l->member[i]);
    printf("(size %d)\n", l->size);
}

int main(void) {
    WaitList l;
    list_init(&l);
    list_insert(&l, 0, 2104); list_insert(&l, 0, 2101);
    list_insert(&l, 1, 2102); list_insert(&l, 3, 2110);
    list_print(&l);                                  /* 2101 2102 2104 2110 (size 4) */
    list_delete(&l, 1);
    list_print(&l);                                  /* 2101 2104 2110 (size 3) */
    printf("insert at 9: %d\n", list_insert(&l, 9, 2120));
    printf("insert at -1: %d\n", list_insert(&l, -1, 5));
    while (l.size < CAPACITY) list_insert(&l, l.size, 9999);
    printf("insert full: %d\n", list_insert(&l, l.size, 8888));
    printf("delete at size: %d\n", list_delete(&l, l.size));
    while (l.size) list_delete(&l, 0);
    printf("delete empty: %d\n", list_delete(&l, 0));
    return 0;
}
```

Line-by-line explanation

Code	Explanation
#define CAPACITY 8	Maximum number of elements; one named constant, easy to change.
typedef struct {...} WaitList;	The representation: an array for the values and a counter of how many are in use.
list_init	Sets size to 0; array contents need not be cleared because only the first size entries are meaningful.
if (l->size == CAPACITY || pos < 0 || pos > l->size) return 0;	Checks both preconditions. Position equal to size is allowed and means “append”.
for (int i = l->size; i > pos; --i) ...	Shifts elements from pos onward one place right, starting from the end, creating a gap.
l->member[pos] = value; ++l->size;	Places the value in the gap and increases the count.

Code	Explanation
if (pos < 0 || pos >= l->size) return 0;	Delete precondition: pos == size is not allowed.
for (int i = pos; i < l->size - 1; ++i)	Shifts later elements one place left, overwriting the deleted element.
list_print	Prints values and size; const shows it does not change the list.
main	Builds a list, deletes an element and tests the failure cases: invalid positions, full list, empty list.

Dry run (trace) of the main program

Notation: the list shows only `member[0..size-1]`.

Step	Call	Condition check	Shifts	List after	Size	Return
1	insert(0, 2104)	size 0 < 8, 0 ≤ 0 ≤ 0	0	2104	1	1
2	insert(0, 2101)	0 ≤ 0 ≤ 1	1	2101 2104	2	1
3	insert(1, 2102)	0 ≤ 1 ≤ 2	1	2101 2102 2104	3	1
4	insert(3, 2110)	0 ≤ 3 ≤ 3 (append)	0	2101 2102 2104 2110	4	1
5	print	–	–	2101 2102 2104 2110	4	–
6	delete(1)	0 ≤ 1 < 4	2	2101 2104 2110	3	1
7	insert(9, 2120)	pos 9 > size 3 → fails	0	unchanged	3	0
8	insert(−1, 5)	pos < 0 → fails	0	unchanged	3	0
9	fill with 9999	five successful appends	0	…	8	1 each
10	insert(8, 8888)	size = CAPACITY → fails	0	unchanged	8	0
11	delete(8)	pos 8 ≥ size 8 → fails	0	unchanged	8	0
12	delete(0) repeatedly	eight successful deletes	–	empty	0	1 each

Step	Call	Condition check	Shifts	List after	Size	Return
13	delete(0)	pos 0 ≥ size 0 → fails	0	empty	0	0

Test plan

No.	Test	Input	Expected	Reason
1	Insert in empty list	insert(0, 2104)	returns 1, size 1	Smallest valid case
2	Insert at front	insert(0, 2101)	order 2101 2104	Shift right works
3	Insert in middle	insert(1, 2102)	2101 2102 2104	Middle position
4	Insert at end (pos = size)	insert(3, 2110)	2101 2102 2104 2110	Upper boundary of valid pos
5	Delete middle	delete(1)	2101 2104 2110	Shift left works
6	Insert position too large	insert(9, 2120)	returns 0, list unchanged	Invalid position
7	Insert negative position	insert(−1, 5)	returns 0	Lower bound violation
8	Insert into full list	insert(size, 8888) at size 8	returns 0	Capacity precondition
9	Delete pos = size	delete(size)	returns 0	One past the end
10	Delete from empty list	delete(0)	returns 0	Empty-structure case

Actual output

Compiled with `gcc -std=c11 -Wall -Wextra -pedantic t3.c -o t3` (no warnings) and run with `./t3`.

Actual output (captured in terminal):

```
2101 2102 2104 2110 (size 4)
2101 2104 2110 (size 3)
insert at 9: 0
insert at -1: 0
insert full: 0
delete at size: 0
delete empty: 0
```



Screenshot 2: Task 3 – compile and run of t3.c (array WaitList)

The output matches every expected value in the test plan (tests 1–10).

Complexity analysis

Let n be the current size and p the position.

Operation	Elements moved	Best case	Worst case	Average (uniform p)
insert at p	n − p	O(1) (p = n)	O(n) (p = 0)	about n/2 moves → O(n)
delete at p	n − p − 1	O(1) (p = n − 1)	O(n) (p = 0)	about (n − 1)/2 moves → O(n)
print	none	Θ(n)	Θ(n)	Θ(n)
size / init	none	Θ(1)	Θ(1)	Θ(1)

For CAPACITY 8 and size 7, insert at position p shifts 7 − p elements: 7, 6, 5, 4, 3, 2, 1, 0 for p = 0…7; the average is 28 / 8 = 3.5 moves. Space is Θ(CAPACITY) plus one integer for size.

Array versus linked-list implementation

Program: ll3.c (linked-list WaitList)

```c
/* ll3.c - linked-list implementation of the same WaitList ADT */
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int member; struct Node *next; } Node;

/* insert value so that it becomes the element at index pos (0 = head) */
int ll_insert(Node **head, int pos, int value) {
    if (pos < 0) return 0;
    Node **link = head;
    for (int i = 0; i < pos; i++) {
        if (*link == NULL) return 0;          /* pos beyond size */
        link = &(*link)->next;
    }
    Node *n = malloc(sizeof *n);
    if (!n) return 0;
    n->member = value; n->next = *link; *link = n;
    return 1;
}
```

```c
int ll_delete(Node **head, int pos) {
    if (pos < 0) return 0;
    Node **link = head;
    for (int i = 0; i < pos; i++) {
        if (*link == NULL) return 0;
        link = &(*link)->next;
    }
    if (*link == NULL) return 0;
    Node *dead = *link; *link = dead->next; free(dead);
    return 1;
}

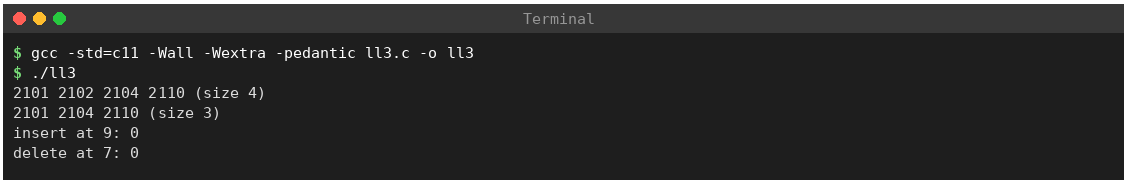
void ll_print(const Node *h) {
    int c = 0;
    for (; h; h = h->next, c++) printf("%d ", h->member);
    printf("(size %d)\n", c);
}

void ll_free(Node *h) { while (h) { Node *t = h->next; free(h); h = t; } }

int main(void) {
    Node *head = NULL;
    ll_insert(&head, 0, 2104); ll_insert(&head, 0, 2101);
    ll_insert(&head, 1, 2102); ll_insert(&head, 3, 2110);
    ll_print(head);
    ll_delete(&head, 1);
    ll_print(head);
    printf("insert at 9: %d\n", ll_insert(&head, 9, 2120));
    printf("delete at 7: %d\n", ll_delete(&head, 7));
    ll_free(head);
    return 0;
}
```

Actual output (captured in terminal):

```
2101 2102 2104 2110 (size 4)
2101 2104 2110 (size 3)
insert at 9: 0
delete at 7: 0
```



Screenshot 3: Task 3 – compile and run of ll3.c (linked-list WaitList)

The first two lines are identical to the array version and the failed insert/delete return 0 as specified.

Property	Array implementation	Linked-list implementation
Reach position k	O(1) (direct index)	O(k) traversal
Insert / delete once at the place	O(n − k) shifting	O(1) pointer change
Insert / delete at position k overall	O(n)	O(k) (traversal dominates)
Extra memory	None per element; unused capacity wasted	One pointer per node plus allocation overhead
Capacity limit	Fixed (CAPACITY)	Limited only by memory
Cache behaviour	Excellent (contiguous)	Poorer (nodes scattered)
Failure modes	Overflow (list full)	malloc failure; leaks if free forgotten

For a small bounded waiting list the array is simpler and faster in practice. The linked list is preferable when the list is long, grows unpredictably and changes happen at positions already reached (for example the head).

Modular implementation (header and source files)

An ADT is best delivered as an interface (`waitlist.h`) and an implementation (`waitlist.c`); the client includes only the header.

File: waitlist.h (interface)

```c
/* waitlist.h - interface of the WaitList ADT */
#ifndef WAITLIST_H
#define WAITLIST_H

#define CAPACITY 8

typedef struct { int member[CAPACITY]; int size; } WaitList;

void list_init(WaitList *l);
int  list_insert(WaitList *l, int pos, int value);
int  list_delete(WaitList *l, int pos);
void list_print(const WaitList *l);
int  list_size(const WaitList *l);

#endif
```

File: waitlist.c (implementation)

```c
/* waitlist.c - array implementation of the WaitList ADT */
#include <stdio.h>
#include "waitlist.h"

void list_init(WaitList *l) { l->size = 0; }

int list_insert(WaitList *l, int pos, int value) {
    if (l->size == CAPACITY || pos < 0 || pos > l->size) return 0;
```

```
    for (int i = l->size; i > pos; --i) l->member[i] = l->member[i - 1];
    l->member[pos] = value;
    ++l->size;
    return 1;
}

int list_delete(WaitList *l, int pos) {
    if (pos < 0 || pos >= l->size) return 0;
    for (int i = pos; i < l->size - 1; ++i) l->member[i] = l->member[i + 1];
    --l->size;
    return 1;
}

void list_print(const WaitList *l) {
    for (int i = 0; i < l->size; i++) printf("%d ", l->member[i]);
    printf("(size %d)\n", l->size);
}

int list_size(const WaitList *l) { return l->size; }
```

File: main3.c (client)

```
/* main3.c - client of the WaitList ADT (uses only the interface) */
#include <stdio.h>
#include "waitlist.h"

int main(void) {
    WaitList w;
    list_init(&w);
    list_insert(&w, 0, 2104);
    list_insert(&w, 0, 2101);
    list_insert(&w, 1, 2102);
    list_print(&w);
    printf("size = %d\n", list_size(&w));
    printf("delete(5) = %d\n", list_delete(&w, 5));
    list_delete(&w, 0);
    list_print(&w);
    return 0;
}
```

Compile and run: `gcc -std=c11 -Wall -Wextra -pedantic main3.c waitlist.c -o waitlist && ./waitlist`

Actual output (captured in terminal):

```
2101 2102 2104 (size 3)
size = 3
delete(5) = 0
2102 2104 (size 2)
```



Screenshot 4: Task 3 – modular build and run (waitlist.h, waitlist.c, main3.c)

The include guard (`#ifndef WAITLIST_H … #endif`) prevents the header from being processed twice. Because the struct is visible in the header, C cannot fully hide the fields; "use only the functions" is a convention. Complete hiding needs an opaque pointer (`typedef struct WaitList WaitList;`) with the struct defined only in the `.c` file, at the price of dynamic allocation.

Common mistakes and how they are avoided

Mistake	Consequence	How the code avoids it
Shifting forward in insert	Overwrites values	Loop runs backwards from size to pos + 1
Allowing pos == size in delete	Reads/writes beyond valid data	Delete uses pos >= size as failure
Forgetting to update size	List and counter disagree	Single place: ++size / --size after the shift
Not checking capacity	Buffer overflow – undefined behaviour	First test in insert
Printing without size	Prints garbage	list_print loops only to size

Viva questions with model answers

1. **What is an ADT?** A set of values with the operations allowed on them, defined independently of the implementation.
2. **Why return 0 instead of crashing on a bad call?** The caller can detect the error and recover; the list stays unchanged and valid.
3. **Time complexity of insert?** O(n) worst case because up to n elements are shifted; O(1) at the end.
4. **Why must the insert loop run from the end?** Running forward would overwrite an element before it is copied.
5. **What changes if we switch to a linked list?** Only the implementation file; the header functions and their meaning stay the same.
6. **What does `const WaitList *l` mean?** The function cannot modify the list through that pointer.

Conclusion

The WaitList ADT defines the ordered waiting members with clearly stated preconditions and postconditions. The array implementation is correct for all tested boundary cases, runs insert and delete in

O(n) because of shifting, and can be replaced by a linked list without changing callers. Separating interface from implementation makes the design maintainable and testable.

TASK 4 – Analyse Array Storage (4 Marks)

To derive the address formulas of one- and two-dimensional arrays, predict the output of a pointer/array program, distinguish fixed, variable-length and malloc-allocated arrays, and analyse insertion and deletion cost.

Let the base address be B and the element size s bytes. - One-dimensional: `address(a[i]) = B + i × s` - Two-dimensional, row-major, `a[R][C]`: `address(a[i][j]) = B + (i × C + j) × s`

Worked examples

Case	Substitution	Address
int a[10], B = 1000, s = 4, element a[7]	1000 + 7 × 4	1028
int a[5][8], B = 2000, s = 4, element a[3][2]	2000 + (3 × 8 + 2) × 4 = 2000 + 104	2104
int m[3][4], m[2][1] relative to m[0][0]	2 × 4 + 1 elements	9 elements (36 bytes)
Library copy: floor 2, shelf 15, position 7	2 × 30,000 + 15 × 100 + 7	index 61,507 (byte offset 246,028)

Three kinds of array storage

Kind	Size known	Storage	Notes
Fixed array int a[100]	Compile time	Automatic (stack) or static	Cannot grow
Variable-length array int a[n]	Run time, at declaration	Automatic (stack)	Optional in C11; large n can overflow the stack
malloc array	Run time, any time	Heap (dynamic)	Must be released with free; can be resized with realloc

When an array is passed to a function, the parameter `int a[]` is really `int *a`: only an address is passed, so the length must be passed separately, and `sizeof a` inside the function gives the size of a pointer.

Predict the output

Program: t4.c

```c
/* t4.c - array storage: address formulas, pointer output, dynamic array doubling */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

static void fill(int *a, int n) { for (int i = 0; i < n; i++) a[i] = i * i; }

int main(void) {
    /* Part 1: predicted-output program */
```

```c
    int a[5] = {9, 9};
    int *p = a + 1;
    printf("%d %d %zu\n", a[0], a[2], sizeof a / sizeof a[0]);   /* 9 0 5 */
    fill(a, 5);
    printf("%d %d %d\n", *p, *(p + 2), p[3]);                    /* 1 9 16 */

    /* Part 2: address arithmetic */
    int b[10];
    int m[3][4];
    printf("&b[7]-&b[0] in bytes = %td\n", (char *)&b[7] - (char *)&b[0]);       /* 28 */
    printf("&m[2][1]-&m[0][0] = %td elements\n", &m[2][1] - &m[0][0]);           /* 9 */
    printf("sizeof m = %zu, sizeof int = %zu\n", sizeof m, sizeof(int));

    /* Part 3: formulas with B=1000, s=4 and B=2000 */
    long B1 = 1000, B2 = 2000, s = 4;
    printf("a[7]    : %ld\n", B1 + 7 * s);                       /* 1028 */
    printf("a5x8[3][2]: %ld\n", B2 + (3 * 8 + 2) * s);           /* 2104 */

    /* Part 4: dynamic array doubling */
    int cap = 1, n = 0;
    long copied = 0;
    int *d = malloc(sizeof(int) * (size_t)cap);
    if (!d) return 1;
    for (int i = 0; i < 10; i++) {
        if (n == cap) {
            cap *= 2;
            copied += n;
            int *t = realloc(d, sizeof(int) * (size_t)cap);
            if (!t) { free(d); return 1; }
            d = t;
            printf("grow: capacity -> %d\n", cap);
        }
        d[n++] = i;
    }
    printf("n=%d capacity=%d elements copied during growth=%ld\n", n, cap, copied);
    free(d);
    return 0;
}
```

Step	State of a[0..4]	Reasoning
int a[5] = {9, 9};	9, 9, 0, 0, 0	First two elements 9; the rest are zero-initialised
int *p = a + 1;	p → a[1]	Pointer arithmetic moves by one element (4 bytes)
first printf	–	a[0] = 9, a[2] = 0, length = 5 → 9 0 5
fill(a, 5)	0, 1, 4, 9, 16	a[i] = i × i
second printf	–	*p = a[1] = 1; *(p+2) = a[3] = 9; p[3] = a[4] = 16 → 1 9 16

Actual output (captured in terminal):

```
9 0 5
1 9 16
&b[7]-&b[0] in bytes = 28
&m[2][1]-&m[0][0] = 9 elements
sizeof m = 48, sizeof int = 4
a[7]    : 1028
a5x8[3][2]: 2104
grow: capacity -> 2
grow: capacity -> 4
grow: capacity -> 8
grow: capacity -> 16
n=10 capacity=16 elements copied during growth=15
```



Screenshot 5: Task 4 – compile and run of t4.c

The first two lines match the prediction; `&m[2][1] - &m[0][0] = 9` confirms the third worked example; 28 bytes confirms a[7] at offset 7 × 4.

Insertion and deletion in the middle

Inserting at position k in an array of n elements shifts n − k elements one place right; deleting shifts n − k − 1 elements left. In the worst case (position 0) about n elements move, so the cost is O(n); at the end it is O(1).

Dynamic array with doubling

When the array is full, `realloc` doubles the capacity. In the output the capacity went 1 → 2 → 4 → 8 → 16 while ten items were appended. The elements copied during growth were 1 + 2 + 4 + 8 = 15, which is less than 2 × 10, so the amortised cost of append is O(1); a single resize costs O(n) in the worst case. Unused capacity can approach half of the allocated space just after a doubling. This is why the **transaction history** (append-only) uses a dynamic array.

Conclusion

Array elements are located by simple arithmetic, giving O(1) access. The price is O(n) middle insertion/deletion and fixed capacity unless `realloc` is used. For the library, direct indexing by copy index is the main advantage.

TASK 5 – Analyse Recursive Solutions (4 Marks)

To predict the output of recursive functions, trace the call stack, derive the running time and stack space by solving recurrences, and compare linear recursion, split (divide-and-conquer) recursion and an iterative loop for counting available copies.

Theory of recursion

A function is recursive if it calls itself. Every correct recursive function has a **base case** (answered directly) and a **recursive case** (a smaller instance of the same problem). Without a base case, or if the argument does not move towards it, the recursion never stops and the program ends with a stack overflow. Each call creates an activation record (stack frame) with parameters, local variables and return address; the maximum number of frames alive together (the depth) decides the stack space.

Aspect	Recursion	Iteration (loop)
Control	Function calls itself	Loop statement
Extra space	Stack frames, O(depth)	Usually O(1)
Clarity	Natural for trees, divide and conquer	Natural for simple sequences
Risk	Stack overflow if depth is large	Infinite loop if condition wrong

Part A – Predict the output

`show(n)` prints n before the call and again after it returns; `mystery(n) = mystery(n/2) + n % 2` counts the 1-bits of n.

Trace of show(3):

Event	Action	Stack (top on right)	Printed so far
1	call show(3)	show(3)	
2	print 3, call show(2)	show(3), show(2)	3
3	print 2, call show(1)	show(3), show(2), show(1)	3 2
4	print 1, call show(0)	…, show(0)	3 2 1
5	show(0) returns (base)	show(3), show(2), show(1)	3 2 1
6	show(1) prints 1, returns	show(3), show(2)	3 2 1 1
7	show(2) prints 2, returns	show(3)	3 2 1 1 2
8	show(3) prints 3, returns	(empty)	3 2 1 1 2 3

Predicted output of show(3): `3 2 1 1 2 3`; calls = n + 1 = 4; depth = 4.

n	Binary	Recursion (innermost first)	Result
5	101	m(0)=0; m(1)=1; m(2)=1; m(5)=1+1	2

n	Binary	Recursion (innermost first)	Result
8	1000	m(0)=0; (1)=1; (2)=1; (4)=1; m(8)=1+0	1
15	1111	m(0)=0; (1)=1; (3)=2; (7)=3; m(15)=3+1	4

Predicted output: `3 2 1 1 2 3` newline `2 1 4`.

Running time of mystery: $T(n) = T(n/2) + c$. Unrolling: $T(n) = T(n/2^k) + k \cdot c$; stops when $n/2^k < 1$, i.e. $k \approx \log_2 n$. By the Master theorem ($a = 1$, $b = 2$, $f(n) = \Theta(1) = \Theta(n^{(\log_b a)})$) $T(n) = \Theta(\log n)$; stack depth $\approx \log_2 n + 1$, space $O(\log n)$. **show(n):** $T(n) = T(n-1) + c = O(n)$, depth $n + 1$, space $O(n)$.

Function	Recurrence	Time	Stack space
show(n)	$T(n) = T(n-1) + c$	$O(n)$	$O(n)$
mystery(n)	$T(n) = T(n/2) + c$	$O(\log n)$	$O(\log n)$

Part B – Counting available copies recursively

The status array has 1 for an issued copy and 0 for an available one. Sample array a[10] = {1,0,0,1,1,0,1,0,1,1} has four available copies (indices 1, 2, 5, 7).

Program: t5.c

```c
/* t5.c - recursion: show, mystery, counting available copies three ways */
#include <stdio.h>

static long calls, depth, maxdepth;

static void show(int n) {
    if (n == 0) return;
    printf("%d ", n);
    show(n - 1);
    printf("%d ", n);
}

static int mystery(int n) { return n == 0 ? 0 : mystery(n / 2) + n % 2; }

/* status[i] = 1 issued, 0 available */
static int count_avail(const int *a, int n) {
    int r;
    calls++; depth++; if (depth > maxdepth) maxdepth = depth;
    r = (n == 0) ? 0 : (a[n - 1] == 0) + count_avail(a, n - 1);
    depth--;
    return r;
}

static int count_avail_split(const int *a, int lo, int hi) {
    int r;
    calls++; depth++; if (depth > maxdepth) maxdepth = depth;
    if (lo > hi) r = 0;
```

```
    else if (lo == hi) r = (a[lo] == 0);
    else {
        int mid = lo + (hi - lo) / 2;
        r = count_avail_split(a, lo, mid) + count_avail_split(a, mid + 1, hi);
    }
    depth--;
    return r;
}

static int count_avail_loop(const int *a, int n) {
    int c = 0;
    for (int i = 0; i < n; i++) if (a[i] == 0) c++;
    return c;
}

/* tail-recursive form with accumulator */
static int count_tail(const int *a, int n, int acc) {
    return n == 0 ? acc : count_tail(a, n - 1, acc + (a[n - 1] == 0));
}

static long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }

static long fib_calls;
static long fib(int n) { fib_calls++; return n < 2 ? n : fib(n - 1) + fib(n - 2); }

static long power(long x, int n) {
    if (n == 0) return 1;
    long h = power(x, n / 2);
    return n % 2 ? h * h * x : h * h;
}

static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

int main(void) {
    show(3); printf("\n");
    printf("%d %d %d\n", mystery(5), mystery(8), mystery(15));

    int a[10] = {1, 0, 0, 1, 1, 0, 1, 0, 1, 1};
    calls = depth = maxdepth = 0;
    int r = count_avail(a, 10);
    printf("linear: %d free, %ld calls, max depth %ld\n", r, calls, maxdepth);
    calls = depth = maxdepth = 0;
    r = count_avail_split(a, 0, 9);
    printf("split : %d free, %ld calls, max depth %ld\n", r, calls, maxdepth);
    printf("loop  : %d free\n", count_avail_loop(a, 10));
    printf("tail  : %d free\n", count_tail(a, 10, 0));

    /* depth for larger n (all slots available, status 0) */
    static int big[10000];
    calls = depth = maxdepth = 0;
```

```
    count_avail(big, 10000);
    printf("linear n=10000: calls %ld, max depth %ld\n", calls, maxdepth);
    calls = depth = maxdepth = 0;
    count_avail_split(big, 0, 9999);
    printf("split  n=10000: calls %ld, max depth %ld\n", calls, maxdepth);
    static int huge[150000];
    calls = depth = maxdepth = 0;
    count_avail_split(huge, 0, 149999);
    printf("split  n=150000: calls %ld, max depth %ld\n", calls, maxdepth);

    printf("fact(5)=%ld\n", fact(5));
    for (int n = 5; n <= 25; n += 5) {
        fib_calls = 0;
        long v = fib(n);
        printf("fib(%d)=%ld calls=%ld\n", n, v, fib_calls);
    }
    printf("power(3,13)=%ld\n", power(3, 13));
    printf("gcd(1071,462)=%d\n", gcd(1071, 462));
    return 0;
}
```

Linear recursion: reduces size n to n − 1. Calls n + 1 = 11 for n = 10; T(n) = T(n−1) + c = O(n); depth n + 1; stack space O(n).

Split recursion: the range lo…hi is halved; results are added. T(n) = 2T(n/2) + c. The call tree has n leaves and n − 1 internal nodes: **2n − 1 = 19 calls** for n = 10. Master theorem (a = b = 2, f = Θ(1)) → Θ(n). Depth $\lceil \log_2 n \rceil + 1$, stack space O(log n).

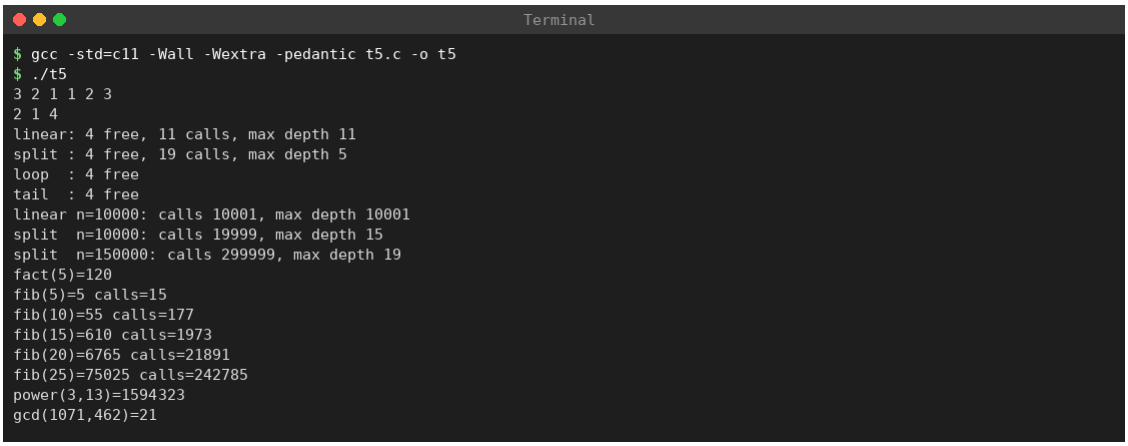
Level	Ranges (n = 10)	Calls
0	[0..9]	1
1	[0..4], [5..9]	2
2	[0..2], [3..4], [5..7], [8..9]	4
3	[0..1], [2..2], [3..3], [4..4], [5..6], [7..7], [8..8], [9..9]	8
4	[0..0], [1..1], [5..5], [6..6]	4
Total		19

Iterative loop: one counter; time O(n), space O(1), no call overhead.

Actual output (captured in terminal):

```
3 2 1 1 2 3
2 1 4
linear: 4 free, 11 calls, max depth 11
split : 4 free, 19 calls, max depth 5
loop  : 4 free
tail  : 4 free
linear n=10000: calls 10001, max depth 10001
split  n=10000: calls 19999, max depth 15
```

```
split  n=150000: calls 299999, max depth 19
fact(5)=120
fib(5)=5 calls=15
fib(10)=55 calls=177
fib(15)=610 calls=1973
fib(20)=6765 calls=21891
fib(25)=75025 calls=242785
power(3,13)=1594323
gcd(1071,462)=21
```



Screenshot 6: Task 5 – compile and run of t5.c (recursion experiments)

The output agrees with the predictions: `3 2 1 1 2 3`, `2 1 4`, 4 available with **11 and 19 calls**, maximum depth 11 versus 5. The measured depths for larger n (10,001 vs 15 at n = 10,000; 19 for the split version at n = 150,000) match n + 1 and $\lceil \log_2 n \rceil + 1$.

Comparison of the three methods

Method	Calls (n = 10)	Calls (n = 150,000)	Time	Auxiliary space
Linear recursion	11	150,001	O(n)	O(n)
Split recursion	19	299,999 (measured)	O(n)	O(log n)
Loop	0	0	O(n)	O(1)

n	Linear depth (n + 1)	Split depth ($\lceil \log_2 n \rceil + 1$)
200	201	9
2,000	2,001	12
150,000	150,001	19 (measured)

With a frame of about 64 bytes (measured for a small recursive function in Task 8 – compiler dependent), linear recursion over 150,000 copies would need about 150,001 × 64 ≈ 9.6 MB, which exceeds the usual 8 MB default Linux stack and so would overflow (this is an estimate; the program was not run at that size). The split version needs 19 frames; the loop needs none.

Tail recursion

`count_avail` is not tail recursive because the addition happens after the call returns. The accumulator form `count_tail(a, n, acc)` makes the recursive call the last action (output `4` above). A compiler *may* turn this into a jump, but the C standard does not guarantee it, so the loop is the portable solution.

Additional recursion examples

The same program also prints factorial, Fibonacci call counts, fast power and gcd (see output above).

n	fib(n)	Calls (measured)
5	5	15
10	55	177
15	610	1,973
20	6,765	21,891
25	75,025	242,785

Naive Fibonacci makes two calls per level and repeats sub-problems: $C(n) = C(n-1) + C(n-2) + 1$, exponential ($\approx 1.618^n$). `power(3, 13) = 1,594,323` uses $T(n) = T(n/2) + c$, i.e. $O(\log n)$; `gcd(1071, 462) = 21` is tail recursive with $O(\log \min(a, b))$ steps.

Recurrence	Solution	Example
$T(n) = T(n-1) + c$	$\Theta(n)$	show, factorial, linear count
$T(n) = T(n/2) + c$	$\Theta(\log n)$	mystery, power, binary search
$T(n) = 2T(n/2) + c$	$\Theta(n)$	split counting
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	merge sort
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(1.618^n)$	naive Fibonacci

Viva questions with model answers

1. **Two parts of a recursive function?** Base case and recursive case.
2. **Why is the output of show(3) symmetric?** One print happens before the call (going down) and one after it returns (coming up).
3. **What does mystery compute?** The number of 1-bits in the binary form of n.
4. **Why does split recursion make 2n − 1 calls?** Its call tree has n leaves and n − 1 internal nodes.
5. **Which version is best for 150,000 copies?** The loop: same O(n) time, O(1) space, no overflow risk.
6. **Is tail recursion always optimised in C?** No; it depends on compiler and optimisation level.

Conclusion

Predicted and actual outputs agree. show is O(n) in time and space, mystery is O(log n), linear recursion needs O(n) stack, split recursion O(log n) stack, and a loop O(1). For 150,000 copies the loop (or split recursion) is the safe choice.

TASK 6 – Count the Steps of an Algorithm (4 Marks)

To count the basic steps executed by four loop patterns, express each in asymptotic notation, and estimate growth for n = 200, 2,000 and 150,000 copies.

Program: t6.c

```c
/* t6.c - step counting for four loop patterns, and growth table */
#include <stdio.h>
#include <math.h>

static long loop_a(int n) { long c = 0; for (int i = 0; i < n; i++) for (int j = 0; j <= i; j++) c++; return c; }
static long loop_b(int n) { long c = 0; for (int i = 1; i < n; i *= 2) c++; return c; }
static long loop_c(int n) { long c = 0; for (int i = 0; i < n; i++) for (int j = 1; j < n; j *= 2) c++; return c; }
static long loop_d(int n) { long c = 0; for (int i = 0; i * i < n; i++) c++; return c; }

int main(void) {
    printf("%ld %ld %ld %ld\n", loop_a(16), loop_b(16), loop_c(16), loop_d(16));
    printf("counted for n=200 : %ld %ld %ld %ld\n", loop_a(200), loop_b(200), loop_c(200), loop_d(200));
    printf("counted for n=2000: %ld %ld %ld %ld\n", loop_a(2000), loop_b(2000), loop_c(2000), loop_d(2000));
    printf("%8s %16s %10s %14s\n", "n", "n^2", "log2 n", "n log2 n");
    int sizes[3] = {200, 2000, 150000};
    for (int i = 0; i < 3; i++) {
        double n = sizes[i];
        printf("%8d %16.0f %10.2f %14.0f\n", sizes[i], n * n, log2(n), n * log2(n));
    }
    printf("n log n growth 2000->150000: %.1f\n", (150000 * log2(150000.0)) / (2000 * log2(2000.0)));
    return 0;
}
```

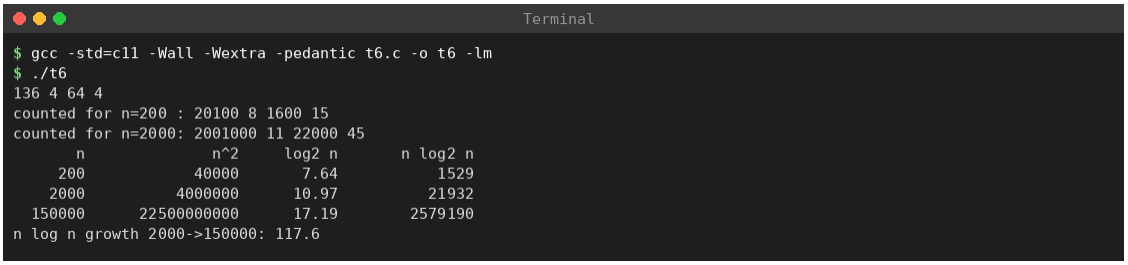
Loop	Counting argument	Count for n = 16	Class
a: for i<n { for j<=i }	inner runs i + 1 times: 1 + 2 + … + n = n(n+1)/2	16 × 17 / 2 = 136	Θ(n²)
b: for (i=1; i<n; i*=2)	i doubles, about $\log_2 n$ iterations	$\log_2 16 = 4$	Θ(log n)
c: for i<n { for (j=1; j<n; j*=2) }	outer n × inner $\log_2 n$	16 × 4 = 64	Θ(n log n)
d: for (i=0; i*i<n; i++)	stops when $i^2 \geq n$: $\lceil\sqrt{n}\rceil$	$\lceil\sqrt{16}\rceil = 4$	Θ(√n)

Predicted: `136 4 64 4`.

Actual output (captured in terminal):

```
136 4 64 4
counted for n=200 : 20100 8 1600 15
```

```
counted for n=2000: 2001000 11 22000 45
       n               n^2      log2 n        n log2 n
     200             40000        7.64            1529
    2000           4000000       10.97           21932
  150000       22500000000       17.19         2579190
n log n growth 2000->150000: 117.6
```



Screenshot 7: Task 6 – compile and run of t6.c (step counts and growth table)

The first line agrees with the prediction. The counted values for n = 200 (20,100; 8; 1,600; 15) and n = 2,000 (2,001,000; 11; 22,000; 45) also match $n(n+1)/2$, $\lceil \log_2 n \rceil$, $n\lceil \log_2 n \rceil$ and $\lceil \sqrt{n} \rceil$.

Meaning of the notations

- **Big-O** gives an upper bound: f = O(g) means f grows no faster than g.
- **Big-Ω** gives a lower bound: f grows at least as fast as g.
- **Big-Θ** gives a tight bound: f is both O(g) and Ω(g).

Growth for the three library sizes

n	**n^2**	**$\log_2 n$**	**$n \log_2 n$**
200	40,000	7.64	1,529
2,000	4,000,000	10.97	21,932
150,000	22,500,000,000	17.19	2,579,190

From 2,000 to 150,000 copies n grows 75 times, n^2 grows 5,625 times, n log n only about 117.6 times, and log n by a factor of 1.6. A quadratic algorithm is unusable for the full library.

Maintained available-copy count

Counting available copies by scanning is Θ(n) every time a display board refreshes. If each floor (and each title) keeps a counter decremented on issue and incremented on return, the display is O(1), each issue/return pays only an extra O(1), and the extra space is O(floors + titles).

Conclusion

Step counting turns loop structure into formulas ($n(n+1)/2$, $\log_2 n$, $n \log_2 n$, $\lceil \sqrt{n} \rceil$). Design choices that turn a Θ(n) display into Θ(1) are essential at n = 150,000.

TASK 7 – Analyse Best, Average and Worst Cases (4 Marks)

To analyse the best-, average- and worst-case number of comparisons of linear search for a book ID, verify it by a program, and compare it with binary search, which needs sorted data.

Theory: why three cases?

Case	Meaning	Use
Best	Input for which the algorithm does the least work	Lower limit; rarely used for decisions
Worst	Input for which it does the most work	A guarantee: never slower
Average	Expected work under a probability assumption	Typical behaviour; needs an assumption

The cost measure is the number of key comparisons.

Problem in the library

The circulation desk enters a 4-digit book code and the system searches the array of codes of the currently issued books. The array holds n = 8 codes in no particular order: {3105, 1170, 9042, 7761, 2288, 6619, 3470, 5503}. Three searches: key at the front (best), key in the middle, key absent (worst).

Linear search – pseudocode

```
for i = 0 to n-1:
    comparisons++
    if a[i] == key: return i
return -1
```

Program: t7.c (linear and binary search with comparison counters)

```c
/* t7.c - best / average / worst case: linear search and binary search on book IDs */
#include <stdio.h>

static int linear(const int *a, int n, int key, int *cmp) {
    *cmp = 0;
    for (int i = 0; i < n; i++) { ++*cmp; if (a[i] == key) return i; }
    return -1;
}

static int binary(const int *a, int n, int key, int *cmp) {
    int lo = 0, hi = n - 1;
    *cmp = 0;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        ++*cmp;
        if (a[mid] == key) return mid;
        if (key < a[mid]) hi = mid - 1; else lo = mid + 1;
    }
    return -1;
}
```

```c
int main(void) {
    int a[8] = {3105, 1170, 9042, 7761, 2288, 6619, 3470, 5503};
    int s[8] = {1170, 2288, 3105, 3470, 5503, 6619, 7761, 9042};
    const char *labels[3] = {"best", "middle", "absent"};
    int keys[3] = {3105, 2288, 1111};
    int cmp, total = 0;

    for (int k = 0; k < 3; k++) {
        int idx = linear(a, 8, keys[k], &cmp);
        printf("linear %-6s key=%d index=%2d comparisons=%d\n", labels[k], keys[k], idx, cmp
);
    }
    for (int i = 0; i < 8; i++) { linear(a, 8, a[i], &cmp); total += cmp; }
    printf("average successful comparisons = %.1f\n", total / 8.0);

    for (int k = 0; k < 3; k++) {
        int idx = binary(s, 8, keys[k], &cmp);
        printf("binary %-6s key=%d index=%2d comparisons=%d\n", labels[k], keys[k], idx, cmp
);
    }
    total = 0;
    printf("binary comparisons per index:");
    for (int i = 0; i < 8; i++) { binary(s, 8, s[i], &cmp); total += cmp; printf(" %d", cmp)
; }
    printf("\nbinary total=%d average=%.3f\n", total, total / 8.0);
    return 0;
}
```

Key	Case	Elements compared in order	Comparisons	Result
3105	Best	3105 (match)	1	index 0
2288	Middle	3105, 1170, 9042, 7761, 2288 (match)	5	index 4
1111	Absent (worst)	all 8 elements, none matches	8	index −1

Actual output (captured in terminal):

```
linear best   key=3105 index= 0 comparisons=1
linear middle key=2288 index= 4 comparisons=5
linear absent key=1111 index=-1 comparisons=8
average successful comparisons = 4.5
binary best   key=3105 index= 2 comparisons=3
binary middle key=2288 index= 1 comparisons=2
binary absent key=1111 index=-1 comparisons=3
binary comparisons per index: 3 2 3 1 3 2 3 4
binary total=21 average=2.625
```



Screenshot 8: Task 7 – compile and run of t7.c (linear and binary search)

The linear results agree with the trace (1, 5, 8 comparisons; average 4.5).

Complexity derivation for linear search

- **Best case:** key at index 0, 1 comparison, $\Theta(1)$.
- **Worst case:** key last or absent, n comparisons, $\Theta(n)$ (8 for n = 8; 150,000 for n = 150,000).
- **Average case (successful, all positions equally likely, probability 1/n):** a key at index i costs i + 1 comparisons, so $E = (1/n) \Sigma (i+1) = (n+1)/2$. For n = 8, E = 4.5, exactly the printed average (1 + 2 + … + 8 = 36; 36 / 8).

If a key is present with probability q and absent otherwise: $E = q \cdot (n+1)/2 + (1 - q) \cdot n$.

q (key present)	n = 8	n = 150,000
1.0	4.5	75,000.5
0.75	0.75 × 4.5 + 0.25 × 8 = 5.375	0.75 × 75,000.5 + 0.25 × 150,000 = 93,750.375
0.5	6.25	112,500.25
0	8	150,000

Case	Comparisons	Class	n = 200	n = 2,000	n = 150,000
Best	1	$\Theta(1)$	1	1	1
Average (success)	(n+1)/2	$\Theta(n)$	100.5	1,000.5	75,000.5
Worst	n	$\Theta(n)$	200	2,000	150,000

The *case* describes the input; O, Ω, Θ describe bounds on a function – they are independent ideas.

Binary search

Binary search needs **sorted** data. It compares the key with the middle element and discards half of the remaining range each time.

```
lo = 0; hi = n-1
while lo <= hi:
    mid = lo + (hi - lo)/2          # avoids overflow of lo+hi
```

```
    if a[mid] == key: return mid
    if key < a[mid]: hi = mid - 1 else lo = mid + 1
return -1
```

Sorted copy: {1170, 2288, 3105, 3470, 5503, 6619, 7761, 9042}. The program's binary lines show: key 3105 (sorted index 2) takes 3 comparisons, key 2288 takes 2, absent key 1111 takes 3 (lo > hi ends the search). *Note:* the true binary best case is the middle element (1 comparison), not the first element.

After k comparisons at most $n/2^k$ elements remain; the worst case is $\lfloor \log_2 n \rfloor + 1$ comparisons, $\Theta(\log n)$.

n	Linear worst	Linear average	Binary worst ($\lfloor \log_2 n \rfloor + 1$)
8	8	4.5	4
200	200	100.5	8
2,000	2,000	1,000.5	11
150,000	150,000	75,000.5	18

For 150,000 copies binary search needs at most 18 comparisons instead of 150,000 – about 8,300 times fewer in the worst case.

Binary comparisons for every position (n = 8, from the program):

Index of key	0	1	2	3	4	5	6	7	Total	Average
Comparisons	3	2	3	1	3	2	3	4	21	2.625

The average 2.625 is far below the linear 4.5; the maximum 4 = $\lfloor \log_2 8 \rfloor + 1$.

The price of keeping the data sorted

Operation	Unsorted array + linear search	Sorted array + binary search
Search	O(n)	O(log n)
Insert a new key	O(1) – append	O(n) – find place and shift
Delete a key	O(n) search + O(1) swap with last	O(n) – shift to close the gap
Preparation	None	Sorting once, O(n log n)

If data change rarely and are searched often (the catalogue of titles) sorting pays off. If data change on every issue/return (the live circulation table) the O(n) maintenance is heavy and a hash table (average O(1) search, insert, delete) is better – as concluded in Task 11.

Structure	Search best	Search average	Search worst	Needs order?
Unsorted array (linear)	O(1)	O(n)	O(n)	No
Sorted array (binary)	O(1)	O(log n)	O(log n)	Yes
Hash table (chaining)	O(1)	O(1) at load factor ≤ 1	O(n) if all keys collide	No

Viva questions with model answers

1. **Why is the worst case often preferred?** It gives a guarantee for all inputs.
2. **Average comparisons of successful linear search?** $(n + 1)/2$.
3. **Worst case of binary search?** $\lfloor \log_2 n \rfloor + 1$ comparisons.
4. **Why can binary search not be used directly on the live circulation array?** It is unsorted and keeping it sorted costs O(n) per update.
5. **Which is better for 150,000 constantly changing records?** A hash table: O(1) average lookup, insert, delete.

Conclusion

The program confirms the theory: linear search takes 1, 5 and 8 comparisons for best, middle and absent keys with average 4.5 = (n+1)/2. Binary search needs at most $\lfloor \log_2 n \rfloor + 1$ comparisons but requires sorted order at O(n) cost per update, so the right structure depends on how often data change compared with how often they are searched.

TASK 8 – Analyse Space Complexity (3 Marks)

To compute the memory needed to store 150,000 copy statuses with an integer array and with a bitmap, and to compare the auxiliary (stack) space of the counting algorithms of Task 5.

Integer array

150,000 × sizeof(int) = 150,000 × 4 = **600,000 bytes ≈ 585.9 KiB** (confirm `sizeof(int)` on the target machine).

Bitmap

A status is only available/issued, so one bit suffices: 150,000 bits / 8 = **18,750 bytes ≈ 18.3 KiB**, which is 32 times smaller. Bit i is in byte `i >> 3` at position `i & 7`. Testing or setting one bit is O(1); counting all by scanning is O(n) (or O(n/8) byte-wise).

Program: t8.c

```c
/* t8.c - space complexity: int array vs bitmap for 150,000 copies; recursion frame size */
#include <stdio.h>
#include <stdint.h>

#define COPIES 150000

static unsigned char bitmap[(COPIES + 7) / 8];

static void set_bit(int i)  { bitmap[i >> 3] |= (unsigned char)(1u << (i & 7)); }
static int  test_bit(int i) { return (bitmap[i >> 3] >> (i & 7)) & 1; }

static uintptr_t last;
static long frame;
static void probe(int n) {
    char c;
    uintptr_t cur = (uintptr_t)&c;
    if (last) frame = (long)(last - cur);
    last = cur;
    if (n > 0) probe(n - 1);
}

int main(void) {
    printf("sizeof(int) = %zu\n", sizeof(int));
    printf("int array bytes = %zu\n", (size_t)COPIES * sizeof(int));
    printf("bitmap bytes    = %zu\n", sizeof bitmap);
    printf("ratio           = %.1f\n", (double)((size_t)COPIES * sizeof(int)) / (double)size
of bitmap);

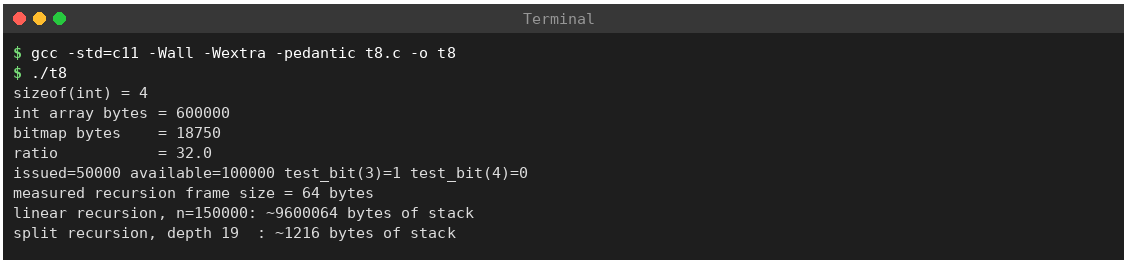
    for (int i = 0; i < COPIES; i += 3) set_bit(i);   /* mark every 3rd copy as issued */
    int issued = 0;
    for (int i = 0; i < COPIES; i++) issued += test_bit(i);
```

```c
    printf("issued=%d available=%d test_bit(3)=%d test_bit(4)=%d\n", issued, COPIES - issued
, test_bit(3), test_bit(4));

    probe(5);
    printf("measured recursion frame size = %ld bytes\n", frame);
    printf("linear recursion, n=150000: ~%ld bytes of stack\n", (long)(COPIES + 1) * frame);
    printf("split recursion, depth 19  : ~%ld bytes of stack\n", 19 * frame);
    return 0;
}
```

Actual output (captured in terminal):

```
sizeof(int) = 4
int array bytes = 600000
bitmap bytes    = 18750
ratio           = 32.0
issued=50000 available=100000 test_bit(3)=1 test_bit(4)=0
measured recursion frame size = 64 bytes
linear recursion, n=150000: ~9600064 bytes of stack
split recursion, depth 19  : ~1216 bytes of stack
```



Screenshot 9: Task 8 – compile and run of t8.c (int array versus bitmap, stack space)

The measured frame size (64 bytes) is for the small `probe` function and is compiler dependent; the 9.6 MB figure is therefore an estimate, not a measurement of `count_avail` itself.

Space of the recursive counters

Method	Time	Auxiliary space	Reason
Linear recursion	O(n)	O(n)	n + 1 frames at maximum depth
Split recursion	O(n)	O(log n)	$\lceil \log_2 n \rceil + 1 = 19$ frames
Simple loop	O(n)	O(1)	A few local variables

Representation (150,000 copies)	Bytes	Test one copy	Count available
int array	600,000	O(1)	O(n) scan, or O(1) with maintained counter
Bitmap	18,750	O(1)	O(n) bit scan, or O(1) with counter

Conclusion

A bitmap uses one thirty-second of the memory of an int array, and recursion that halves the problem uses logarithmic stack instead of linear stack. Space is measured separately from time.

TASK 9 – Implement and Test the Library Data Structures (6 Marks)

To implement a working array-based structure for one shelf of N = 20 copy slots with operations init, issue, return, lookup and a maintained available count, to prove it correct with test cases (including boundary cases and a ten-event sequence), and to analyse its complexity.

No.	Requirement	Source
R1	Store, for each copy slot, whether it is issued and to which member	Task 1 data
R2	Issue a copy to a member and return the slot (−1 when none available)	Task 1 operations
R3	Free a slot on return; reject invalid or already-available slots	Task 1 invariants
R4	Find the slot of a copy borrowed by a member (−1 if none)	Task 1 operations
R5	Keep the available count correct and give it in O(1)	Task 6 recommendation
R6	Be tested for empty shelf, full shelf, first/last slot and a sequence of events	Assignment checklist

Design

Index i of the arrays is the slot number, so the slot is implicit and access by slot is O(1). A member ID needs 16 bytes (15 characters + `\0`). Size: 20 × 4 (issued) + 20 × 16 (member) + 4 (count) = 80 + 320 + 4 = **404 bytes** (confirmed by `sizeof(Shelf)` in the output).

Invariants: (1) `avail_count` = number of i with `issued[i] == 0`; (2) 0 ≤ `avail_count` ≤ N; (3) if `issued[i] == 0` then `member[i]` is the empty string; (4) no double issue – issue marks the slot issued before returning it and never selects an issued slot.

Algorithms (pseudocode)

```
init:   for all i: issued[i]=0, member[i]=""; avail_count = N
issue:  for i = 0..N-1: if not issued[i]: mark issued, copy member ID, avail_count--, return
i
        return -1                                       # none available
return: if slot<0 or slot>=N or not issued[slot]: return 0
        issued[slot]=0; member[slot]=""; avail_count++; return 1
lookup: for i = 0..N-1: if issued[i] and member[i]==id: return i;  return -1
```

Program: t9.c

```c
/* t9.c - array-based library shelf: init, issue, return, lookup, maintained count */
#include <stdio.h>
#include <string.h>

#define N 20   /* copy slots on one shelf */

typedef struct {
```

```
    int  issued[N];        /* 1 = issued, 0 = available */
    char member[N][16];    /* borrower ID of each issued copy ("" if available) */
    int  avail_count;      /* maintained number of available copies */
} Shelf;

static void init(Shelf *p) {
    for (int i = 0; i < N; i++) { p->issued[i] = 0; p->member[i][0] = '\0'; }
    p->avail_count = N;
}

/* returns copy slot, or -1 when no copy is available */
static int issue_copy(Shelf *p, const char *member) {
    for (int i = 0; i < N; i++) {
        if (!p->issued[i]) {
            p->issued[i] = 1;
            strncpy(p->member[i], member, 15);
            p->member[i][15] = '\0';
            --p->avail_count;
            return i;
        }
    }
    return -1;
}

/* returns 1 on success, 0 for invalid or already-available slot */
static int return_copy(Shelf *p, int slot) {
    if (slot < 0 || slot >= N || !p->issued[slot]) return 0;
    p->issued[slot] = 0;
    p->member[slot][0] = '\0';
    ++p->avail_count;
    return 1;
}

/* slot of the first copy borrowed by member, or -1 */
static int lookup(const Shelf *p, const char *member) {
    for (int i = 0; i < N; i++)
        if (p->issued[i] && strcmp(p->member[i], member) == 0) return i;
    return -1;
}

static int tests = 0, fails = 0;
#define CHECK(cond, msg) do { tests++; if (!(cond)) { fails++; printf("FAIL: %s\n", msg); }
else printf("PASS: %s\n", msg); } while (0)

static int consistent(const Shelf *p) {
    int f = 0;
    for (int i = 0; i < N; i++) f += !p->issued[i];
    return f == p->avail_count;
}
```

```c
int main(void) {
    Shelf p; init(&p);
    int a = issue_copy(&p, "M2301"), b = issue_copy(&p, "M2302");
    printf("issued to slots %d and %d\n", a, b);
    printf("lookup = %d\n", lookup(&p, "M2302"));
    printf("available copies = %d\n", p.avail_count);
    printf("return = %d\n", return_copy(&p, a));
    printf("available copies = %d\n", p.avail_count);
    printf("return again = %d\n", return_copy(&p, a));
    printf("invalid slot = %d\n", return_copy(&p, N));
    printf("---- additional tests\n");

    Shelf q; init(&q);
    CHECK(q.avail_count == N, "T1 empty shelf has N available");
    CHECK(lookup(&q, "NONE") == -1, "T2 lookup on empty shelf fails");
    CHECK(return_copy(&q, 0) == 0, "T3 return of available slot fails");
    char id[16];
    for (int i = 0; i < N; i++) { sprintf(id, "M%04d", i); issue_copy(&q, id); }
    CHECK(q.avail_count == 0, "T4 full shelf has 0 available");
    CHECK(issue_copy(&q, "M9999") == -1, "T5 issue on full shelf fails");
    CHECK(lookup(&q, "M0000") == 0, "T6 first slot lookup");
    CHECK(lookup(&q, "M0019") == N - 1, "T7 last slot lookup");
    CHECK(return_copy(&q, N - 1) == 1 && q.avail_count == 1, "T8 return last slot");
    CHECK(issue_copy(&q, "M5000") == N - 1, "T9 freed slot is reused");
    CHECK(return_copy(&q, -1) == 0, "T10 negative slot rejected");

    /* ten-event sequence */
    Shelf s; init(&s);
    int r1 = issue_copy(&s, "V1");          /* 1 */
    int r2 = issue_copy(&s, "V2");          /* 2 */
    int r3 = issue_copy(&s, "V3");          /* 3 */
    int ok4 = return_copy(&s, r2);          /* 4 */
    int r5 = issue_copy(&s, "V4");          /* 5 */
    int ok6 = return_copy(&s, r1);          /* 6 */
    int r7 = issue_copy(&s, "V5");          /* 7 */
    int l8 = lookup(&s, "V2");              /* 8 */
    int l9 = lookup(&s, "V4");              /* 9 */
    CHECK(r1 == 0 && r2 == 1 && r3 == 2 && ok4 && r5 == 1 && ok6 && r7 == 0,
          "T11 ten-event sequence slot assignments");
    CHECK(s.avail_count == 17 && consistent(&s), "T12 counter consistent after sequence");
    CHECK(l8 == -1 && l9 == 1, "T13 returned member absent, new borrower found");

    printf("sizeof(Shelf) = %zu\n", sizeof(Shelf));
    printf("tests=%d fails=%d\n", tests, fails);
    return fails != 0;
}
```

Explanation of the code

Part	Explanation
#define N 20	Number of slots; one constant so arrays and loops agree.
issue_copy	Linear search for the first available slot (lowest index). strncpy(..., 15) copies at most 15 characters and the next statement forces \0, because strncpy does not terminate when the source is 15 characters or longer. --avail_count keeps the counter correct.
return_copy	Checks three failure conditions (negative, ≥ N, not issued) in one if using short-circuit ||; issued[slot] is read only after the range is known valid.
lookup	Compares stored ID with strcmp; issued[i] && avoids matching leftover text.
CHECK macro	Counts tests, counts failures, prints PASS/FAIL; do { } while (0) makes the macro behave like one statement.
consistent	Recounts available slots from scratch and compares with avail_count – an invariant checker.
return fails != 0;	Exit status 0 when all tests pass, 1 otherwise.

Dry run of the sample sequence

Step	Operation	Slot 0	Slot 1	Slot 2	avail_count	Return
0	init	–	–	–	20	–
1	issue("M2301")	M2301	–	–	19	0
2	issue("M2302")	M2301	M2302	–	18	1
3	lookup("M2302")	M2301	M2302	–	18	1
4	return(0)	–	M2302	–	19	1 (success)
5	return(0) again	–	M2302	–	19	0 (already available)
6	return(20)	–	M2302	–	19	0 (invalid slot)

Test plan and results

Test	What is tested	Expected result	Actual
Sample	Two issues → slots 0, 1; lookup 1; available 18; return succeeds; available 19; repeat fails; slot N fails	As in the dry run	PASS (see output)

Test	What is tested	Expected result	Actual
T1	Empty shelf	avail_count = 20	PASS
T2	Lookup on empty shelf	−1	PASS
T3	Return of available slot	0	PASS
T4	Fill all 20 slots	avail_count = 0	PASS
T5	Issue on full shelf	−1	PASS
T6	Lookup of first slot	0	PASS
T7	Lookup of last slot	19 (N − 1)	PASS
T8	Return last slot	1; avail_count = 1	PASS
T9	Issue again after freeing	reuses slot 19	PASS
T10	Negative slot number	0	PASS
T11	Ten-event sequence slot assignments	V1→0, V2→1, V3→2, V4→1, V5→0	PASS
T12	Counter consistency after the sequence	avail_count = 17 and recount agrees	PASS
T13	Returned member absent, new member found	lookup(V2) = −1, lookup(V4) = 1	PASS

The ten-event sequence (T11–T13):

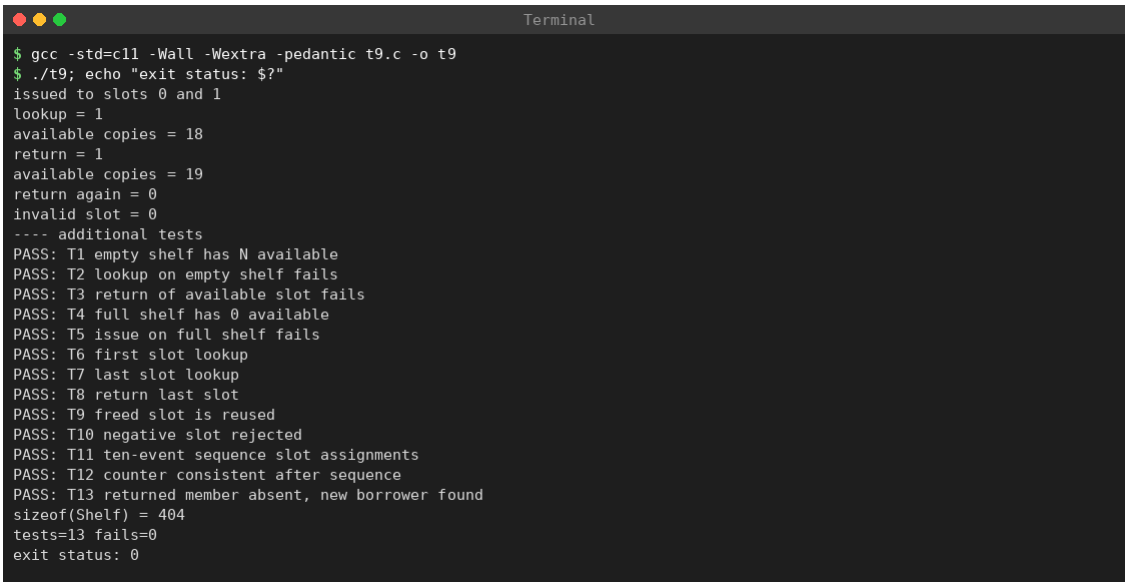
Event	Operation	Slots issued after	Result
1	issue V1	0	slot 0
2	issue V2	0, 1	slot 1
3	issue V3	0, 1, 2	slot 2
4	return slot of V2	0, 2	success
5	issue V4	0, 1, 2	slot 1 (lowest free index)
6	return slot of V1	1, 2	success
7	issue V5	0, 1, 2	slot 0
8	lookup V2	0, 1, 2	−1 (V2 returned)
9	lookup V4	0, 1, 2	1
10	check avail_count	3 issued	17 available

Actual output

Compiled with `gcc -std=c11 -Wall -Wextra -pedantic t9.c -o t9` – no warnings; exit status 0.

Actual output (captured in terminal):

```
issued to slots 0 and 1
lookup = 1
available copies = 18
return = 1
available copies = 19
return again = 0
invalid slot = 0
---- additional tests
PASS: T1 empty shelf has N available
PASS: T2 lookup on empty shelf fails
PASS: T3 return of available slot fails
PASS: T4 full shelf has 0 available
PASS: T5 issue on full shelf fails
PASS: T6 first slot lookup
PASS: T7 last slot lookup
PASS: T8 return last slot
PASS: T9 freed slot is reused
PASS: T10 negative slot rejected
PASS: T11 ten-event sequence slot assignments
PASS: T12 counter consistent after sequence
PASS: T13 returned member absent, new borrower found
sizeof(Shelf) = 404
tests=13 fails=0
```



Screenshot 10: Task 9 – compile and run of t9.c with exit status 0 (all tests pass)

The first seven lines are the sample sequence and match the dry run; all 13 additional tests pass (`tests=13 fails=0`).

Complexity analysis

Operation	Best	Worst	Space	Reason
init	Θ(N)	Θ(N)	O(1)	Visits every slot once
issue	O(1)	O(N)	O(1)	First slot free vs. last slot free / full
return	O(1)	O(1)	O(1)	Direct index and counter update
lookup	O(1)	O(N)	O(1)	Linear search with string compare
available count	O(1)	O(1)	O(1)	Maintained counter

For N = 20 the loops are trivial. At 150,000 slots issue and lookup would take up to 150,000 steps, which motivates the hash table and free-slot index of Tasks 11–12. Total storage grows as N × 20 + 4 bytes: 3,000,004 bytes (≈ 2.86 MiB) for N = 150,000.

Improved version: free-slot stack and duplicate check

Two improvements are implemented for N = 8: a stack of free slot numbers gives `issue` an O(1) way to find a slot, and a lookup before issuing rejects a member who already holds the copy (return codes ≥ 0 slot, −1 none available, −2 duplicate).

Program: t9b.c

```c
/* t9b.c - improved shelf: free-slot stack (O(1) find) and duplicate-borrower check */
#include <stdio.h>
#include <string.h>

#define N 8

typedef struct {
    int  issued[N];
    char member[N][16];
    int  free_stack[N];   /* stack of available slot numbers */
    int  top;             /* entries on the stack */
    int  avail_count;
} Shelf;

static void init(Shelf *p) {
    for (int i = 0; i < N; i++) { p->issued[i] = 0; p->member[i][0] = '\0'; }
    for (int i = 0; i < N; i++) p->free_stack[i] = N - 1 - i;   /* slot 0 on top */
    p->top = N; p->avail_count = N;
}

static int lookup(const Shelf *p, const char *m) {
    for (int i = 0; i < N; i++) if (p->issued[i] && strcmp(p->member[i], m) == 0) return i;
    return -1;
}

/* returns slot, -1 no copy available, -2 member already holds a copy of this title */
```

```c
static int issue_copy(Shelf *p, const char *m) {
    if (lookup(p, m) != -1) return -2;
    if (p->top == 0) return -1;
    int s = p->free_stack[--p->top];
    p->issued[s] = 1;
    strncpy(p->member[s], m, 15); p->member[s][15] = '\0';
    --p->avail_count;
    return s;
}

static int return_copy(Shelf *p, int s) {
    if (s < 0 || s >= N || !p->issued[s]) return 0;
    p->issued[s] = 0; p->member[s][0] = '\0';
    p->free_stack[p->top++] = s; ++p->avail_count;
    return 1;
}

int main(void) {
    Shelf p; init(&p);
    printf("issue A = %d\n", issue_copy(&p, "A"));
    printf("issue B = %d\n", issue_copy(&p, "B"));
    printf("issue A again = %d\n", issue_copy(&p, "A"));
    printf("return slot 0 = %d\n", return_copy(&p, 0));
    printf("issue C = %d\n", issue_copy(&p, "C"));
    char id[8];
    for (int i = 0; i < 6; i++) { sprintf(id, "X%d", i); issue_copy(&p, id); }
    printf("available=%d top=%d\n", p.avail_count, p.top);
    printf("issue to full shelf = %d\n", issue_copy(&p, "Z"));
    return 0;
}
```

Actual output (captured in terminal):

```
issue A = 0
issue B = 1
issue A again = -2
return slot 0 = 1
issue C = 0
available=0 top=0
issue to full shelf = -1
```



Screenshot 11: Task 9 – compile and run of t9b.c (free-slot stack and duplicate check)

Property	Basic version	Improved version
Finding a free slot	Scan, O(N)	Pop from stack, O(1)
Duplicate check	Not done	lookup, O(N)
issue total time	O(N)	O(N) because of the duplicate check (O(1) with a hash table)
return	O(1)	O(1)
Extra memory	None	N + 1 integers
Slot chosen	Lowest free index	Most recently freed slot (LIFO)

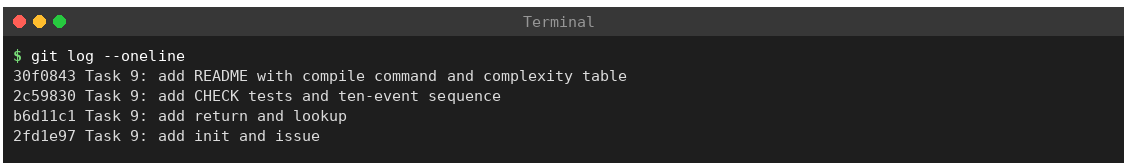
The output shows: A and B at slots 0 and 1; the second issue to A refused (−2); after slot 0 is freed, C receives slot 0 (LIFO); after six more issues the shelf is full (`available=0 top=0`) and the next issue returns −1.

Debugging notes

Symptom	Likely cause	Fix
Garbage characters when printing an ID	Missing \0 after strncpy	Set member[i][15] = '\0'
avail_count negative or above N	Counter changed without checking the slot state	Update only after a successful state change
Lookup finds a returned book	ID not cleared on return	Clear the string and test issued[i]
Signed/unsigned warning	Mixed types	Use consistent int/size_t; compile with -Wall -Wextra

Version-control plan

1. Create the GitHub repository (for example `smart-library-ds`).
2. Commit 1: `t9.c` with init and issue — “Task 9: add init and issue”. 3. Commit 2: add return and lookup. 4. Commit 3: add the CHECK tests and the sequence test. 5. Commit 4: add README.md with compile command, expected/actual output and complexity table.



Screenshot 12: Task 9 – git log –oneline showing the four commits of the version-control plan

Viva questions with model answers

1. **Why keep `avail_count`?** Counting is O(N) on every display; a maintained counter is O(1) and costs O(1) per issue/return.
2. **Why is the slot number not stored?** It is the array index: O(1) access and no memory.

3. **Why does `issue` force the terminating character?** `strncpy` does not terminate when the source is at least as long as the limit.
4. **Worst case of `issue`?** Shelf full or only the last slot free: N checks.
5. **Two boundary tests?** First and last slot; empty and full shelf.
6. **How to stop one member borrowing the same copy twice?** Call `lookup` first and refuse if it returns a slot.

Conclusion

The array-based Shelf structure meets all requirements for 20 slots: O(1) return and counting, O(N) issue and lookup. The sample sequence and 13 additional tests – empty, full, first-slot, last-slot and ten-event cases – all pass and keep the count consistent. For 150,000 copies the same logic must be combined with a free-slot index and a hash table (Tasks 11–12).

TASK 10 – Compare Theory with Measurement (4 Marks)

To measure the running time of operations on arrays of 200, 2,000, 150,000 and 1,000,000 elements and compare the measurements with the theoretical complexity.

Method

- The same compiler and flags for every size: `gcc -std=c11 -Wall -Wextra -pedantic` (**no optimisation**, so the loops are not removed).
- Each operation is repeated (100 times for scan and lookup, 20 times for insert/delete) so the total is large enough for the timer.
- `clock()` returns CPU time; the result is divided by `CLOCKS_PER_SEC`. A `volatile` sink stops the compiler deleting the loop.
- Insert-at-front: each repetition shifts the whole array right by one (`memmove`) and back, so it contains two shifts per repetition. Delete-at-front: one left shift per repetition.
- The program was run **three times**; the table below shows the first run, and the growth analysis uses the mean of the three runs.

Program: t10.c

```c
/* t10.c - measure scan, lookup, front insert and front delete on arrays of increasing size
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static volatile long sink;

static double secs(clock_t a, clock_t b) { return (double)(b - a) / CLOCKS_PER_SEC; }

int main(void) {
    int sizes[4] = {200, 2000, 150000, 1000000};
    printf("%8s %12s %12s %12s %12s\n", "n", "scan(100x)", "lookup(100x)", "insert0(20x)", "
delete0(20x)");
    for (int s = 0; s < 4; s++) {
        int n = sizes[s];
        int *a = malloc(sizeof(int) * (size_t)(n + 1));
        if (!a) return 1;
        for (int i = 0; i < n; i++) a[i] = i;

        clock_t t0 = clock();
        long sum = 0;
        for (int r = 0; r < 100; r++) for (int i = 0; i < n; i++) sum += a[i];
        clock_t t1 = clock();
        sink = sum;
```

```
        long found = -1;
        for (int r = 0; r < 100; r++) { for (int i = 0; i < n; i++) if (a[i] == n - 1) { fou
nd = i; break; } }
        clock_t t2 = clock();
        sink = found;

        for (int r = 0; r < 20; r++) {      /* insert at front, then undo (two shifts) */
            memmove(a + 1, a, sizeof(int) * (size_t)n);
            memmove(a, a + 1, sizeof(int) * (size_t)n);
        }
        clock_t t3 = clock();

        for (int r = 0; r < 20; r++) memmove(a, a + 1, sizeof(int) * (size_t)(n - 1));
        clock_t t4 = clock();
        sink = a[0];

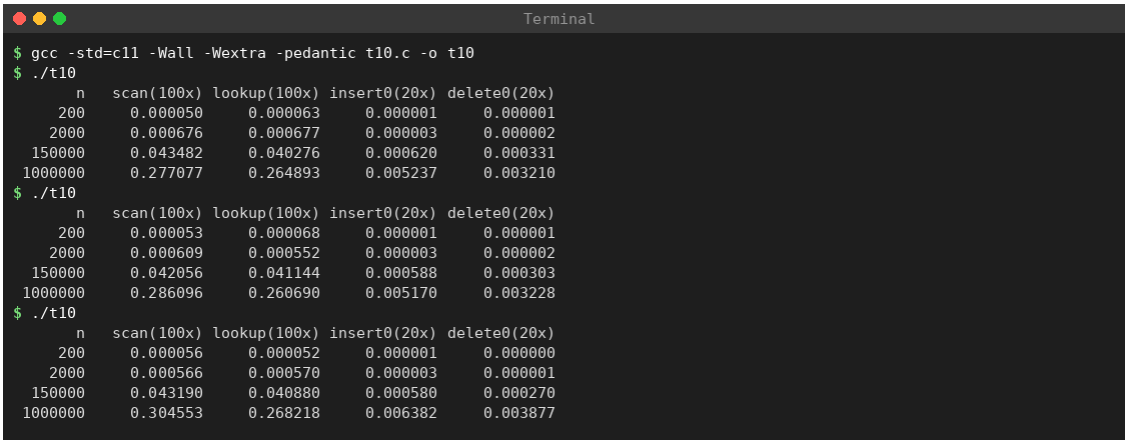
        printf("%8d %12.6f %12.6f %12.6f %12.6f\n", n, secs(t0, t1), secs(t1, t2), secs(t2,
t3), secs(t3, t4));
        free(a);
    }
    return 0;
}
```

Measured results

Machine: Intel Xeon @ 2.10 GHz, 2 CPUs, 7.8 GiB RAM, gcc 13.3.0, Linux. These are genuine measurements taken on this machine; absolute times depend on the CPU, so they will differ on another computer, but the growth pattern should not.

Actual output (run 1, times in seconds):

```
       n   scan(100x) lookup(100x) insert0(20x) delete0(20x)
     200     0.000050     0.000063     0.000001     0.000001
    2000     0.000676     0.000677     0.000003     0.000002
  150000     0.043482     0.040276     0.000620     0.000331
 1000000     0.277077     0.264893     0.005237     0.003210
```



Screenshot 13: Task 10 – compile and three runs of t10.c (timings in seconds)

Mean of three runs (seconds):

n	scan (100×)	lookup (100×)	insert0 (20×)	delete0 (20×)
200	0.000053	0.000061	0.000001	0.000001
2,000	0.000617	0.000600	0.000003	0.000002
150,000	0.042909	0.040767	0.000596	0.000301
1,000,000	0.289242	0.264600	0.005596	0.003438

Comparison with theory

Step in n	Theory: Θ(n) scan grows by	Measured scan growth (mean of 3 runs)
200 → 2,000	10.0×	0.000617 / 0.000053 ≈ 11.6×
2,000 → 150,000	75.0×	0.042909 / 0.000617 ≈ 69.5×
150,000 → 1,000,000	6.7×	0.289242 / 0.042909 ≈ 6.7×

At n = 1,000,000 one element costs about 0.289242 / (100 × 1,000,000) ≈ 2.9 nanoseconds (unoptimised code).

For large n the measured growth is very close to the predicted linear growth, so the scan is Θ(n). For small n the timer resolution (about a microsecond) and start-up effects dominate, so the ratio is less accurate (for example the 200 → 2,000 step grew by 11.6× instead of the predicted 10×). Front insertion/deletion also grows with n because every element moves, but `memmove` copies whole blocks, so the cost per element is much lower than for the scan loop. The lookup (target in the last position, so effectively a full scan with a comparison) behaves like the scan.

Counting operations versus timing

Method	Advantage	Disadvantage
Counting basic operations	Exactly reproducible; independent of machine	Ignores cache, compiler and constant factors
Measuring with clock()	Shows real CPU time	Noisy; depends on CPU, optimisation, cache and timer resolution

Conclusion

Measured behaviour confirms the theory for large n: scan and lookup scale linearly, and front insertion/deletion cost grows with the number of elements moved. Theory predicts the trend; measurement reveals the constants.

TASK 11 – Recommend and Defend the Final Design (5 Marks)

To choose the final data-structure design for the largest scenario (150,000 copies), compare it with the alternatives of Tasks 1–10, and defend it with complexity analysis, memory estimates and measured evidence.

Requirements that drive the design

Requirement	Number / rule	Consequence for the design
Number of copies	n = 150,000	O(n) work on every event is too slow; O(1) or O(log n) needed
Issue / return / status update	Very frequent	Access by copy index must be O(1)
Availability display	Continuous	Maintained counter per floor/title, O(1)
Lookup by book/copy ID	Frequent	Index on the ID (not just an array scan)
Waiting list	When a title is fully issued	Per-title FIFO queue (priority queue in Task 12)
Overdue retrieval	Daily, most overdue first	Max-heap on days overdue (Task 12)
Invariants (Task 1)	No double issue, correct counts	All structures updated through one controlled function

Candidate designs

Code	Design	Lookup by ID	Issue / return	Most-overdue query
A	One flat array, linear scans for everything	O(n)	O(1) by index (find a free copy O(n))	O(n)
B	Array kept sorted by ID + binary search	O(log n)	O(n) (shifting)	O(n)
C	Per-floor dynamic arrays + hash table + counters + queues + overdue heap (**recommended**)	O(1) average	O(1) array + O(1) hash + O(log n) heap	O(1) peek
D	Balanced BST (AVL) by ID + arrays	O(log n) worst	O(log n)	O(n) unless extra index
E	Linked list of issued copies	O(n)	O(1) at head; delete needs search	O(n)

Weighted decision matrix

Scores run from 1 (poor) to 5 (excellent). The weights and scores are the author's judgement based on the complexities above; they show the reasoning and are not measurements.

Criterion (weight)	A	B	C	D	E
Book lookup by ID (25)	1	4	5	4	1
Issue / return update (25)	5	1	4	4	3
Overdue / priority query (20)	1	2	5	3	1
Memory economy (10)	5	5	3	3	3
Simplicity (10)	5	3	2	2	4
Worst-case guarantee (10)	1	4	3	5	1
Weighted total (maximum 500)	**280**	**285**	**405**	**360**	**200**

Example calculation for design C: 25×5 + 25×4 + 20×5 + 10×3 + 10×2 + 10×3 = 125 + 100 + 100 + 30 + 20 + 30 = 405. Design C scores highest; its main weaknesses are memory and complexity, which are judged acceptable (see the memory estimate below).

Recommended design

Per-floor dynamic arrays of Copy records behind a Copy ADT; a hash table keyed by copy/book ID; maintained available counts per floor and per title; a singly linked borrowing list per member; a FIFO (priority, Task 12) reservation queue per title; a dynamic array for the transaction history; and a max-heap for overdue loans.

Component	What	Why
1. Per-floor dynamic arrays behind a Copy ADT	Each floor owns a malloc-ed array of Copy records	O(1) access by index (Task 4); floors may differ in size (Task 2); invariants enforced in one place (Task 3)
2. Hash table by copy/book ID	Array of buckets, each a chain of (ID, slot) nodes	"Where is this book?" and "is it already issued?" become O(1) average instead of O(n) (a scan averages 75,000.5 comparisons)
3. Maintained counts	One integer per floor and per title, ±1 inside issue/return	Display in O(1) (Task 6)
4. Member borrowing list	Singly linked list per member	Few items, delete without shifting
5. Reservation queue	FIFO queue per title	O(1) enqueue/dequeue
6. Transaction history	Dynamic array, doubling	O(1) amortised append (Task 4)
7. Overdue max-heap	Heap keyed by days overdue	Most overdue record in O(1) peek, O(log n) insert/remove

Complexity of every operation

Operation	Recommended design C	Plain array (A)	Sorted array (B)
Update status by copy index	O(1)	O(1)	O(1) if indexed
Issue	O(1) hash insert + O(1) free-slot pop	O(n) scan	O(n) shift
Return	O(1) array + O(1) hash delete	O(n) to find the copy	O(n) shift
Lookup by ID	O(1) average, O(n) worst	O(n)	O(log n)
Availability display	O(1)	O(n)	O(n)
Add to waiting list	O(1)	–	–
Most overdue loan	O(1) peek	O(n)	O(n)
Monthly report	O(n) – infrequent	O(n)	O(n)

Hash table analysis

Load factor α = keys / buckets; with chaining the expected chain length is α and a successful lookup examines about $1 + \alpha/2$ nodes. Choosing $\alpha = 0.75$ for 150,000 copies: buckets = 150,000 / 0.75 = 200,000, rounded to the prime **200,003**.

Method	Idea	Advantage	Disadvantage
Separate chaining (chosen)	Each bucket holds a list of colliding keys	Simple deletion; works for $\alpha > 1$	Pointer per node, extra allocation
Open addressing	Next free bucket in the array	Contiguous, no pointers	Clustering; deletion needs tombstones; needs $\alpha < 1$

Structure	Lookup	Insert / delete	Extra space
Hash table (average)	O(1)	O(1)	O(n)
Sorted array + binary search	O(log n) guaranteed	O(n)	O(1)
Linear search	O(n)	O(1) at end / O(n) in middle	O(1)

Evidence: measured behaviour of the hash table

The program builds a chained hash table for 150,000 copy IDs with 200,003 buckets (djb2 hash), then looks up every copy. IDs are generated deterministically, so results are reproducible. It counts string comparisons (basic operations), not time.

Program: t11.c

```c
/* t11.c - chained hash table for 150,000 book copies; counts string comparisons */
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>

#define V 150000
#define B 200003            /* about V / 0.75, rounded to a prime */

typedef struct Node { char reg[16]; int slot; struct Node *next; } Node;

static char regs[V][16];
static Node pool[V];
static Node *tab[B];

static unsigned long hash(const char *s) {          /* djb2 */
    unsigned long h = 5381;
    while (*s) h = h * 33 + (unsigned char)*s++;
    return h;
}

int main(void) {
    for (int i = 0; i < V; i++)
        sprintf(regs[i], "LB%02d%c%06d", i % 97, 'A' + i % 26, i);   /* unique copy IDs */

    for (int i = 0; i < V; i++) {
        unsigned long h = hash(regs[i]) % B;
        Node *nd = &pool[i];
        strcpy(nd->reg, regs[i]); nd->slot = i; nd->next = tab[h]; tab[h] = nd;
    }

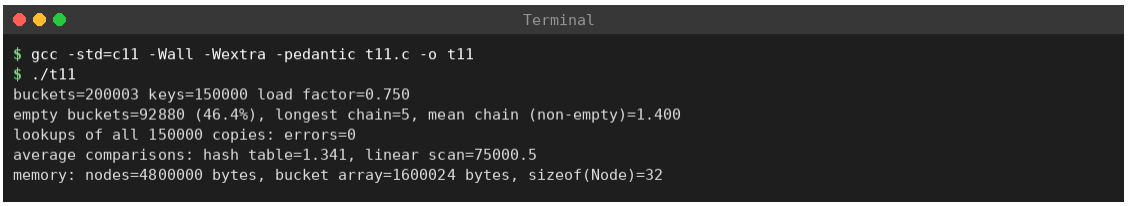
    int empty = 0, maxc = 0; long long chain_sum = 0;
    for (int b = 0; b < B; b++) {
        int c = 0;
        for (Node *p = tab[b]; p; p = p->next) c++;
        if (c == 0) empty++;
        if (c > maxc) maxc = c;
        chain_sum += c;
    }
    printf("buckets=%d keys=%d load factor=%.3f\n", B, V, (double)V / B);
    printf("empty buckets=%d (%.1f%%), longest chain=%d, mean chain (non-empty)=%.3f\n",
           empty, 100.0 * empty / B, maxc, (double)chain_sum / (B - empty));

    long long hash_cmp = 0, lin_cmp = 0; int errors = 0;
    for (int i = 0; i < V; i++) {
        Node *p = tab[hash(regs[i]) % B]; int c = 0, ok = 0;
        while (p) { c++; if (strcmp(p->reg, regs[i]) == 0) { ok = 1; break; } p = p->next; }
        hash_cmp += c; if (!ok) errors++;
        lin_cmp += i + 1;
    }
    printf("lookups of all %d copies: errors=%d\n", V, errors);
    printf("average comparisons: hash table=%.3f, linear scan=%.1f\n", (double)hash_cmp / V,
(double)lin_cmp / V);
```

```
    printf("memory: nodes=%zu bytes, bucket array=%zu bytes, sizeof(Node)=%zu\n",
           (size_t)V * sizeof(Node), (size_t)B * sizeof(Node *), sizeof(Node));
    return 0;
}
```

Actual output (captured in terminal):

```
buckets=200003 keys=150000 load factor=0.750
empty buckets=92880 (46.4%), longest chain=5, mean chain (non-empty)=1.400
lookups of all 150000 copies: errors=0
average comparisons: hash table=1.341, linear scan=75000.5
memory: nodes=4800000 bytes, bucket array=1600024 bytes, sizeof(Node)=32
```



Screenshot 14: Task 11 – compile and run of t11.c (hash table with 150,000 copies)

Measure	**Measured**	**Theory (Poisson model, α = 0.750)**
Empty buckets	46.4 % (92,880 of 200,003)	e^(−α) = 47.2 %
Mean chain length of non-empty buckets	1.400	α / (1 − e^(−α)) = 1.42
Average comparisons per successful lookup (hash)	1.341	1 + α/2 = 1.375
Average comparisons per lookup (linear scan)	75000.5	(n + 1)/2 = 75,000.5
Longest chain	5	small; grows very slowly with n
Lookup errors	0	0 (correctness)

Interpretation: the hash table needs about 1.3 comparisons per lookup, whereas the linear scan needs 75,000.5 – a reduction of roughly 55,929 times in basic operations. The measured values are close to the theoretical model (the keys hash slightly better than the random model predicts), which justifies the O(1) average-lookup assumption. The longest chain (5) shows the worst case is far from O(n) for this key set.

Memory estimate of the recommended design

Component	**Calculation**	**Bytes**
Copy records	150,000 × 16 bytes (assumed record: ID, status/location, due date, member index)	2,400,000
Hash nodes	150,000 × 32 bytes (measured sizeof(Node) = 32)	4,800,000

Component	Calculation	Bytes
Hash bucket array	200,003 × 8-byte pointers	1,600,024
Overdue heap index	150,000 entries × 8 bytes (days + loan index), worst case all loans overdue	1,200,000
Counters, queues, history	a few integers per floor/title; queue and history sizes depend on use	not included
Total (without queues/history)		**10,000,024 (about 9.54 MiB)**

This is about 16.7 times the 600,000-byte int array of Task 8, but 9.5 MiB is tiny for a modern server, so the memory cost is acceptable in exchange for O(1) lookups and fast overdue queries. If memory were scarce, status could be a bitmap (18,750 bytes, Task 8) and nodes could be made smaller.

Risks and their control

Risk	Effect	Control
Structures disagree (array, hash, heap, counters)	Double issue or wrong count	Only issue/return modify them, in a fixed order; invariant checker in tests
Many hash collisions	Lookup approaches O(n)	Good hash function; load factor ≤ 0.75; rehash when exceeded
Rehash pause	One slow operation, O(n)	Pre-size for 150,000 copies; rehash off-peak
Allocation failure	Issue fails	Check every malloc; pre-allocate pools
Over-engineering for a small library	Needless complexity	Use the simple array for a prototype (Task 12)

Defence of the design with evidence

Claim	Evidence	Status
Hash lookup is O(1) on average	1.341 comparisons per lookup vs 75,000.5 for a scan (output above)	Obtained
Array update by slot is O(1)	Direct index; Task 9 tests; Task 4 address formula	Obtained
Available count is O(1)	Counter; Task 9 consistency test T12	Obtained
Heap gives most-overdue loan cheaply	Operation counts and 0 mismatches against the scan (Task 12)	Obtained (operation counts)
Latency of the whole design	Time scan vs indexed implementation at n = 200, 2,000, 150,000	**Not measured** – operation counts only

Why the other designs were rejected

- **A (plain array):** correct and simplest, but every lookup, count and overdue query costs O(n) – up to 150,000 steps. Prototype only.
- **B (sorted array):** fast lookup, but every issue and return shifts up to n elements; one array cannot be sorted by ID and by overdue days simultaneously.
- **D (balanced tree):** guaranteed O(log n) lookup and ordered traversal, a good alternative if the worst-case guarantee matters more than speed; more complex than hashing.
- **E (linked list):** O(n) search and no direct indexing; useful only for short lists and queues.

Peak-hour workload model

Assumed peak hour (assumptions made for this exercise – replace with real logs): 600 issues, 600 returns, 1,500 "where is this book?" lookups and 3,600 display refreshes for n = 150,000. The unit is basic operations (one slot check or one key comparison); figures are estimates from the analysis and the measured hash average, **not timings**.

Activity	Count	Design A ops each	Design A total	Design C ops each	Design C total
Issue (find an available copy)	600	150,000.0	90,000,000	2.341 (free-slot pop + hash insert)	1,405
Return (find copy by ID)	600	75,000.5	45,000,300	2.341 (hash delete + stack push)	1,405
Lookup by ID	1,500	75,000.5	112,500,750	1.341 (hash lookup)	2,012
Display refresh	3,600	150,000.0	540,000,000	1.000 (read counter)	3,600
Total			**787,501,050**		**8,421**

Design C needs roughly 8,421 operations in the peak hour against about 787,501,050 for the plain array – a ratio of about 93,520 : 1. Real running-time ratios will be much smaller (hash operations touch scattered memory), but the model shows where the effort goes: in design A about 69 % of all operations are spent only on counting available copies for display boards, which the maintained counter reduces to a single read. If the library had only n = 200 copies the difference would be small, which supports using the simple array for a prototype.

Possible extensions

Replace the binary heap by an indexed priority queue if eligibility changes often; persist records in a database for monthly summaries; shard by floor so floors are processed in parallel.

Viva questions with model answers

1. **Why per-floor dynamic arrays?** O(1) access by index and flexible floor sizes.
2. **How many buckets for 150,000 copies at α = 0.75?** 150,000 / 0.75 = 200,000, rounded to a prime such as 200,003.
3. **Worst case of hashing?** O(n) when all keys fall into one bucket.
4. **Why not a sorted array for the live data?** Updates cost O(n) because of shifting.
5. **Main danger of several indexes?** They can become inconsistent; all updates go through one function and are tested.

Conclusion

For 150,000 copies the best balance is design C: slot arrays for O(1) updates, a hash table for average O(1) lookup, maintained counters for O(1) display, queues for reservations and a heap for overdue loans. The measured hash behaviour (1.341 comparisons per lookup against 75,000.5 for a scan) supports the choice. The price is about 9.5 MiB of memory and more complex update code, controlled by routing all changes through one ADT and testing the invariants. Final latency claims must wait for timing measurements.

TASK 12 – Re-evaluate and Adapt the Design (4 Marks)

Three new requirements arrive after the first design: **priority reservations** for faculty and research scholars, **search by title, author or subject** as well as by ID, and **overdue fine tracking** in which the most overdue records are retrieved first. The aim is to (A) analyse the impact, (B) evaluate whether the original array design still works, (C) implement and test a revised function, and (D) defend the adapted design with evidence.

Part A – Impact analysis

Requirement	New or changed data	Why needed
Priority reservations	Member category (faculty / scholar / student) on each reservation; arrival number	Serve faculty first, then scholars, then students; FIFO within a class
Search by title / author / subject	Indexes: title (sorted or hash), author → books, subject → books	ID-only lookup cannot answer catalogue queries
Overdue fine tracking	Due date, days overdue, fine rate per loan; an ordered index by days overdue	Most overdue first without scanning every loan

Operation	Change
Add to waiting list	Insert with key (class, arrival number) – plain FIFO is no longer enough
Serve waiting list	Remove the minimum (class, arrival)
Catalogue search	New: by title (binary search / hash), author and subject (hash → list of books)
Overdue retrieval	New, frequent: remove the maximum days overdue
Return	Also updates the overdue index and computes the fine

Part B – Evaluate the original design

The original design is a plain array of loans. To find the most overdue active loan it must examine every loan, skip returned ones and remember the largest days value: a scan of all n records, **O(n) per query**.

n	Checks per query (scan)	Checks for 1,000 queries
200	200	200,000
2,000	2,000	2,000,000
150,000	150,000	150,000,000

The scan is correct and easy to validate, but it does not scale: the original design is adequate for a prototype only. A plain FIFO queue also cannot give faculty priority without scanning the queue.

Part C – Revised functions

Program: t12.c (reference scan, with a faculty-only variant)

```c
/* t12.c - most-overdue record by linear scan (reference version), with faculty filter */
#include <stdio.h>

#define N 10

typedef struct { int active, faculty, days; } Loan;   /* active = still unreturned */

/* index of the active loan with the largest days overdue, or -1.  only_faculty restricts to
faculty loans */
int most_overdue(const Loan a[], int n, int only_faculty) {
    int best = -1;
    for (int i = 0; i < n; i++) {
        if (!a[i].active) continue;
        if (only_faculty && !a[i].faculty) continue;
        if (best == -1 || a[i].days > a[best].days) best = i;
    }
    return best;
}

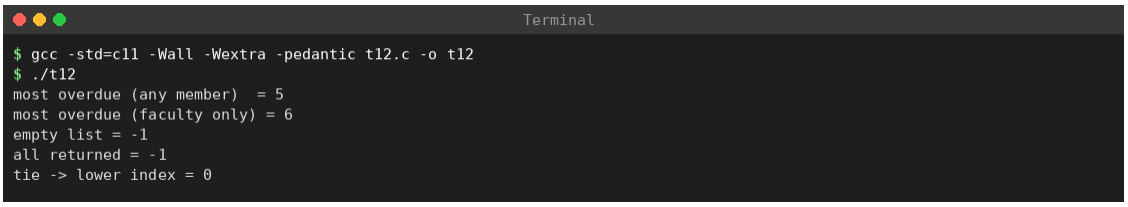
int main(void) {
    Loan a[N] = {{1,0,12},{0,0,40},{1,1,4},{1,0,7},{0,1,30},
                 {1,0,33},{1,1,5},{0,0,6},{1,0,9},{1,0,8}};
    printf("most overdue (any member)  = %d\n", most_overdue(a, N, 0));
    printf("most overdue (faculty only) = %d\n", most_overdue(a, N, 1));
    printf("empty list = %d\n", most_overdue(a, 0, 0));
    Loan none[2] = {{0,0,5},{0,1,9}};
    printf("all returned = %d\n", most_overdue(none, 2, 0));
    Loan tie[3] = {{1,0,7},{1,0,7},{1,0,3}};
    printf("tie -> lower index = %d\n", most_overdue(tie, 3, 0));
    return 0;
}
```

Statement	Meaning
int best = -1;	Index of the best loan so far; −1 means none found
if (!a[i].active) continue;	Returned loans are never candidates
if (only_faculty && !a[i].faculty) continue;	Optional filter for faculty loans
if (best == -1 || a[i].days > a[best].days) best = i;	Take the first candidate, then replace it when a larger value is found; || short-circuits so a[best] is read only after best is valid

Actual output (captured in terminal):

```
most overdue (any member)  = 5
most overdue (faculty only) = 6
empty list = -1
```

```
all returned = -1
tie -> lower index = 0
```



Screenshot 15: Task 12 – compile and run of t12.c (most overdue loan by scan)

On the sample data (slots 1, 4 and 7 returned) the most overdue active loan is index 5 (33 days); among faculty loans it is index 6 (5 days). The empty list and all-returned cases return −1, and a tie returns the lower index (strict >).

No.	Situation	Expected	Result
1	Sample data, any member	index 5	PASS
2	Sample data, faculty only	index 6	PASS
3	n = 0	−1	PASS
4	All loans returned	−1	PASS
5	Two loans with equal days	lower index	PASS

Part D – Adapted design with indexes

Overdue loans: a binary **max-heap** keyed by days overdue; the root is the most overdue loan. Peek O(1); remove/insert O(log n). **Reservations:** a binary **min-heap** keyed by (class, arrival number). **Search:** titles kept sorted (binary search, O(log n); the catalogue changes rarely) and author/subject hash tables chaining to book indexes (O(1) average).

Operation	Scan / FIFO (original)	Indexed design
Most overdue loan (peek)	O(n)	O(1)
Remove most overdue	O(n)	O(log n)
Add loan to overdue index	O(1)	O(log n)
Serve next reservation with priority	O(q) scan of the queue	O(log q)
Search by author / subject	O(n)	O(1) average + result size
Extra memory	none	8 bytes per heap entry: 1,200,000 B for 150,000 loans

Experiment: scan versus heap

For n = 200, 2,000 and 150,000 loans the program runs n/10 queries on both structures (every query services the most overdue loan; every third query is followed by a new overdue loan appearing), compares the days value found by the scan with the one from the heap on every query, and counts basic operations (slot checks for the scan; key comparisons for the heap). The initial heap build is not counted. The same pseudo-random data are used for both methods.

Program: t12b.c

```
/* t12b.c - scan vs max-heap for "most overdue record first"; counts basic operations */
#include <stdio.h>
#include <stdlib.h>

typedef struct { int days; int id; } Item;

static Item *heap;
static int hn;
static long long hops;

static void swap_items(Item *a, Item *b) { Item t = *a; *a = *b; *b = t; }

static void sift_up(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        hops++;
        if (heap[p].days < heap[i].days) { swap_items(&heap[p], &heap[i]); i = p; } else bre
ak;
    }
}

static void sift_down(int i) {
    for (;;) {
        int l = 2 * i + 1, r = l + 1, b = i;
        if (l < hn) { hops++; if (heap[l].days > heap[b].days) b = l; }
        if (r < hn) { hops++; if (heap[r].days > heap[b].days) b = r; }
        if (b == i) break;
        swap_items(&heap[i], &heap[b]); i = b;
    }
}

static void hpush(Item it) { heap[hn] = it; sift_up(hn++); }
static Item hpop(void) { Item t = heap[0]; heap[0] = heap[--hn]; sift_down(0); return t; }

static unsigned long long seed = 12345;
static int rnd(void) { seed = seed * 6364136223846793005ULL + 1442695040888963407ULL; return
(int)((seed >> 33) % 365) + 1; }

static void run(int n) {
```

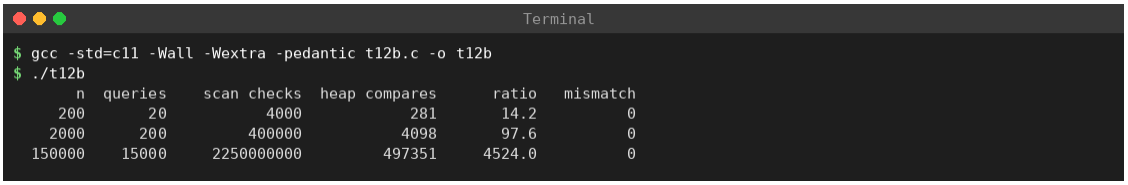
```
    int *days = malloc(sizeof(int) * (size_t)n);
    char *active = malloc((size_t)n);
    heap = malloc(sizeof(Item) * (size_t)(n + 1));
    if (!days || !active || !heap) exit(1);
    hn = 0;
    for (int i = 0; i < n; i++) { days[i] = rnd(); active[i] = 1; Item it = {days[i], i}; hp
ush(it); }
    hops = 0;

    int queries = n / 10, mismatches = 0;
    long long checks = 0;
    for (int q = 1; q <= queries; q++) {
        int best = -1;
        for (int i = 0; i < n; i++) {                 /* O(n) scan */
            checks++;
            if (active[i] && (best == -1 || days[i] > days[best])) best = i;
        }
        Item top = hpop();                            /* O(log n) heap */
        if (top.days != days[best]) mismatches++;
        best = top.id;                                /* service the record the heap chose (
ties may differ) */
        active[best] = 0;
        if (q % 3 == 0) {                             /* a new overdue loan appears in that
slot */
            days[best] = rnd(); active[best] = 1;
            Item it = {days[best], best}; hpush(it);
        }
    }
    printf("%8d %8d %14lld %14lld %10.1f %10d\n", n, queries, checks, hops,
           (double)checks / (double)(hops ? hops : 1), mismatches);
    free(days); free(active); free(heap);
}

int main(void) {
    printf("%8s %8s %14s %14s %10s %10s\n", "n", "queries", "scan checks", "heap compares",
"ratio", "mismatch");
    run(200); run(2000); run(150000);
    return 0;
}
```

Actual output (captured in terminal):

```
       n  queries     scan checks  heap compares      ratio   mismatch
     200       20            4000            281       14.2          0
    2000      200          400000           4098       97.6          0
  150000    15000      2250000000         497351     4524.0          0
```



Screenshot 16: Task 12 – compile and run of t12b.c (scan versus max-heap)

n	Queries	Scan checks	Heap comparisons	Scan / heap ratio	Mismatches
200	20	4,000	281	≈ 14.2×	0
2,000	200	400,000	4,098	≈ 97.6×	0
150,000	15,000	2,250,000,000	497,351	≈ 4524.0×	0

Interpretation: - **Correctness:** for all 15,220 queries the heap returned a loan with exactly the same days-overdue value as the scan (0 mismatches). The heap is a valid replacement. - **Work:** the scan does n checks per query (200, 2,000, 150,000); the heap does about 14, 20 and 33 comparisons per query, growing like log n, so the ratio rises with n. - **Limits of this evidence:** these are operation counts, not times. A heap comparison touches scattered memory while the scan reads sequentially, so the real speed ratio will be smaller. “The indexed version is faster” must be confirmed with timing measurements (Task 10 method) on the same machine.

Priority reservations and catalogue search – demonstration

Program: t12c.c

```c
/* t12c.c - priority reservation queue (faculty first) and search by title / author / subject */
#include <stdio.h>
#include <string.h>

/* ---------- part 1: priority reservation queue (min-heap on class, then arrival order) ---------- */
typedef struct { int cls; int seq; char name[16]; } Req;   /* cls: 0 faculty, 1 research scholar, 2 student */

static Req h[32];
static int hn = 0, seqc = 0;

static int less(const Req *a, const Req *b) { return a->cls < b->cls || (a->cls == b->cls && a->seq < b->seq); }
static void swp(Req *a, Req *b) { Req t = *a; *a = *b; *b = t; }

static void push(int cls, const char *name) {
    Req r; r.cls = cls; r.seq = seqc++;
    strncpy(r.name, name, 15); r.name[15] = '\0';
    h[hn] = r;
```

```c
    int i = hn++;
    while (i > 0) { int p = (i - 1) / 2; if (less(&h[i], &h[p])) { swp(&h[i], &h[p]); i = p;
} else break; }
}

static Req pop(void) {
    Req top = h[0];
    h[0] = h[--hn];
    int i = 0;
    for (;;) {
        int l = 2 * i + 1, r = l + 1, s = i;
        if (l < hn && less(&h[l], &h[s])) s = l;
        if (r < hn && less(&h[r], &h[s])) s = r;
        if (s == i) break;
        swp(&h[i], &h[s]); i = s;
    }
    return top;
}

/* ---------- part 2: search indexes ---------- */
#define NB 8
#define HB 7
typedef struct { const char *title, *author, *subject; } Book;
static const Book books[NB] = {            /* kept sorted by title for binary search */
    {"Algorithms", "Cormen", "CS"},      {"Compilers", "Aho", "CS"},
    {"Data Structures", "Weiss", "CS"},  {"Databases", "Silberschatz", "CS"},
    {"Discrete Maths", "Rosen", "Maths"},{"Networks", "Tanenbaum", "CS"},
    {"Operating Systems", "Silberschatz", "CS"}, {"Statistics", "Walpole", "Maths"}
};

static unsigned long djb2(const char *s) { unsigned long x = 5381; while (*s) x = x * 33 + (
unsigned char)*s++; return x; }

static int head_a[HB], next_a[NB], head_s[HB], next_s[NB];

static int find_title(const char *t) {
    int lo = 0, hi = NB - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2, c = strcmp(t, books[mid].title);
        if (c == 0) return mid;
        if (c < 0) hi = mid - 1; else lo = mid + 1;
    }
    return -1;
}

int main(void) {
    printf("-- reservation queue --\n");
    push(2, "Student-A"); push(0, "Faculty-X"); push(2, "Student-B");
    push(1, "Scholar-P"); push(0, "Faculty-Y"); push(2, "Student-C");
    while (hn) { Req r = pop(); printf("serve %s (class %d)\n", r.name, r.cls); }
```

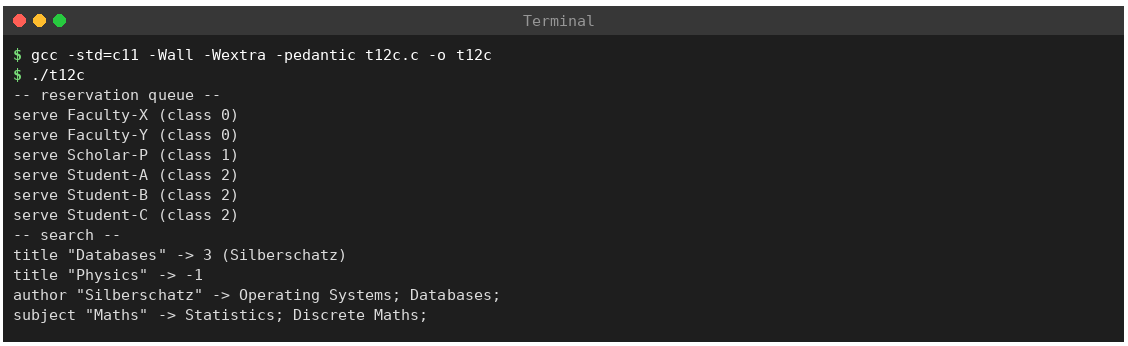
```
    for (int i = 0; i < HB; i++) { head_a[i] = -1; head_s[i] = -1; }
    for (int i = 0; i < NB; i++) {
        unsigned long ha = djb2(books[i].author) % HB, hs = djb2(books[i].subject) % HB;
        next_a[i] = head_a[ha]; head_a[ha] = i;
        next_s[i] = head_s[hs]; head_s[hs] = i;
    }

    printf("-- search --\n");
    int t = find_title("Databases");
    printf("title \"Databases\" -> %d (%s)\n", t, t >= 0 ? books[t].author : "-");
    printf("title \"Physics\" -> %d\n", find_title("Physics"));

    printf("author \"Silberschatz\" ->");
    for (int i = head_a[djb2("Silberschatz") % HB]; i != -1; i = next_a[i])
        if (strcmp(books[i].author, "Silberschatz") == 0) printf(" %s;", books[i].title);
    printf("\nsubject \"Maths\" ->");
    for (int i = head_s[djb2("Maths") % HB]; i != -1; i = next_s[i])
        if (strcmp(books[i].subject, "Maths") == 0) printf(" %s;", books[i].title);
    printf("\n");
    return 0;
}
```

Actual output (captured in terminal):

```
-- reservation queue --
serve Faculty-X (class 0)
serve Faculty-Y (class 0)
serve Scholar-P (class 1)
serve Student-A (class 2)
serve Student-B (class 2)
serve Student-C (class 2)
-- search --
title "Databases" -> 3 (Silberschatz)
title "Physics" -> -1
author "Silberschatz" -> Operating Systems; Databases;
subject "Maths" -> Statistics; Discrete Maths;
```



Screenshot 17: Task 12 – compile and run of t12c.c (priority reservations and search)

Faculty requests are served before the scholar and the students even though a student arrived first, and students are served in arrival order. Author "Silberschatz" returns two titles and subject "Maths" two titles; an absent title returns −1.

Complete adapted design

Part	Original design	Adapted design
Copy record	status flag	status, location, due date, member category
Lookup by ID	Linear scan	Hash table by ID
Available count	Scan or counter	Counter per floor and title
Overdue retrieval	Scan O(n)	Max-heap, O(1) peek
Reservations	FIFO queue	Min-heap on (class, arrival)
Catalogue search	None	Sorted titles + author/subject hash indexes
Update rule	Change array only	One function updates array, hash, counters, heaps together

Evidence-based defence

1. **Keep arrays for a small prototype.** For n = 200 the scan costs only 200 checks per query and its code is shortest and easiest to prove correct.
2. **Modify the design for the new requirements.** For n = 150,000 a scan costs 150,000 checks per query; the heap needed about 33 comparisons per query in the experiment.
3. **Identical queries and data** were used for both methods; 0 mismatches in all 15,220 queries.
4. **Report real latency, memory and correctness.** Correctness: 0 mismatches. Memory: 8 bytes per indexed loan (1,200,000 B). Latency: **to be measured** (Task 10 method).
5. **Do not claim the indexed version is faster until it is timed.** Operation counts are strong evidence, but only a timing test proves speed on the target machine.

Risks of the adapted design

Risk	Control
Heap and loan array disagree	Update both in one function; use the scan as reference in tests (as in the experiment)
Wrong class served first	Reservation key (class, arrival) tested with the demonstration above
Loan returned while still in the heap	Remove on return, or mark inactive and skip lazily
Deleting an arbitrary loan from a heap	Keep a position index per loan, or mark inactive and skip
Extra code makes defects likely	Table-driven tests for every new rule (Task 9 method)

How the max-heap works

A binary heap is a complete binary tree stored in an array in which every parent is not smaller (max-heap) than its children. For index i: parent = ⌊(i − 1)/2⌋, left child = 2i + 1, right child = 2i + 2; the largest element is at index 0. **Insert (sift-up):** place at the end, swap with the parent while larger – at most ⌊$\log_2$ n⌋ swaps. **Remove maximum (sift-down):** move the last element to the root, swap with the larger child while smaller than it – at most ⌊$\log_2$ n⌋ levels. The experiment's `sift_up` and `sift_down` implement exactly this, and each comparison increments `hops`.

Why about 33 comparisons per query at n = 150,000? A heap of 150,000 entries has ⌊$\log_2$ 150,000⌋ + 1 = 18 levels. Remove-maximum compares two children on up to 17 levels (about 2 × 17 = 34 comparisons) and an insertion makes up to 17; the measured mean of 33.2 comparisons per query agrees with this bound and grows with log n, not n.

Viva questions with model answers

1. **What new data do priority reservations require?** The member category and an arrival number.
2. **Complexity of the scan for the most overdue loan?** O(n) per query; 150,000 checks for the largest library.
3. **Why is the scan still useful?** It is simple, correct, and serves as a reference to test the faster index.
4. **Why must the heap stay consistent?** If a returned loan stays in the heap the system may report it again.
5. **Can we claim the indexed version is faster?** Only after a timing measurement; operation counts alone are not proof of time.
6. **Where is the most overdue loan stored in a max-heap?** At index 0.

Conclusion

The new requirements add member categories, catalogue indexes and a frequent "most overdue first" query. The original array design answers it correctly with an O(n) scan (150,000 checks at the largest size). A max-heap index returned identical results in all 15,220 test queries and needed far fewer basic operations (497,351 comparisons against 2.25 billion slot checks at n = 150,000); a min-heap on (class, arrival) gives priority reservations, and hash/sorted indexes give catalogue search. The recommendation is to **keep arrays for the prototype and modify the design (add the indexes) for production**, subject to timing measurements that confirm the speed-up.